

PROMETEO



Honeypot SSH con Cowrie: Análisis de Inteligencia de Amenazas

Proyecto Intermodular ASIR (PIASIR)
Ciclo Formativo de Grado Superior
Administración de Sistemas Informáticos en Red
Modalidad Online

Alumno: Diego Pérez Fonterosa
Centro: Prometeo FP (thePower Education)
Curso: 2025–2026

Primera entrega parcial · Octubre de 2026

*A mi familia, por su apoyo incondicional durante todo el ciclo formativo.
A la comunidad de software libre, sin cuya labor este proyecto no sería
posible.*

Índice de Contenidos

Resumen.....	5
Justificación del Proyecto.....	7
Motivación.....	7
Relación con los Módulos Profesionales.....	7
Estado de la Cuestión.....	9
Comparativa Razonada.....	10
Introducción.....	12
Evolución respecto al Anteproyecto.....	12
Funciones Principales.....	13
Requisitos del Sistema.....	13
Objetivos.....	15
Objetivo General.....	15
Objetivos Específicos.....	15
Desglose RFTP.....	16
Descripción del Sistema.....	20
Estado de los Componentes.....	20
Arquitectura General.....	20
Flujo de Datos.....	21
Seguridad y Aislamiento.....	22
Casos de Uso.....	22
Diseños.....	24
Diagrama de Arquitectura de Red.....	24
Diseño del Flujo de Datos.....	25
Diseño de Interfaces.....	26
Diagrama de Red.....	27
Tecnología.....	29
Cowrie.....	29
Hetzner Cloud.....	29
Python 3.....	30
Wazuh 4.9.2.....	31
MITRE ATT&CK.....	31
Herramientas de Red y Seguridad.....	31
Herramientas de Desarrollo y Documentación.....	32
Metodología.....	34
Metodología Utilizada.....	34
Planificación Temporal.....	34
Presupuesto.....	35
README y Git.....	36
Catálogo de Técnicas MITRE ATT&CK.....	36
Implementación.....	39
Estrategia de Despliegue.....	39
Bastionado del Servidor.....	40
Instalación y Configuración de Cowrie.....	41
Exposición del Honeypot.....	44
Script de Análisis.....	44

Clasificación Automática en MITRE ATT&CK.....	45
Herramienta de Aprovisionamiento en OCI.....	46
Normalización de Saltos de Línea.....	47
Pruebas.....	48
Entorno de Pruebas Local.....	48
Prueba de Arranque.....	48
Incidencia: Rechazo Silencioso de Todos los Logins.....	48
Ataque Simulado.....	49
Validación del Analizador con Datos Reales.....	50
Prueba del Clasificador MITRE ATT&CK.....	50
Prueba de la Herramienta de Aprovisionamiento.....	52
Prueba de la Reserva de Memoria.....	53
Despliegue en Hetzner Cloud.....	53
Pruebas Pendientes en el Servidor.....	56
Resultados.....	57
Volumen de la Actividad.....	57
Naturaleza Automatizada de los Ataques.....	57
Origen Geográfico.....	58
Credenciales.....	59
Campañas Identificadas.....	60
Técnicas MITRE ATT&CK.....	61
Primeras Conclusiones Defensivas.....	63
Trabajos Futuros.....	64
Evolución del Honeypot.....	64
Mejora del Análisis.....	64
Integración y Automatización.....	65
Conclusiones.....	66
Conclusiones Técnicas.....	66
Conclusiones sobre la Infraestructura.....	66
Conclusiones Formativas.....	67
Valoración Personal.....	68
Referencias.....	70
Anexos.....	72
Anexo A. Configuración de Cowrie (cowrie.cfg).....	72
Anexo B. Credenciales del Honeypot (userdb.txt).....	74
Anexo C. Configuración del SSH de Administración.....	75
Anexo D. Servicio de systemd de Cowrie.....	76
Anexo E. Reserva de Memoria (oci-keepalive).....	77
Anexo F. Script de Bastionado (01_hardening.sh).....	79
Anexo G. Script de Instalación de Cowrie (02_instalar_cowrie.sh).....	82
Anexo H. Script de Redirección (03_redireccion_22.sh).....	85

Resumen

Este proyecto aborda el despliegue y análisis de un honeypot SSH de media interacción basado en Cowrie, expuesto a Internet desde un servidor cloud de Hetzner. El sistema simula un servidor Linux vulnerable para atraer ataques automatizados —principalmente intentos de fuerza bruta, ejecución de comandos y descarga de malware— con el objetivo de estudiar las tácticas, técnicas y procedimientos (TTPs) empleados por los atacantes. Los datos capturados durante un período mínimo de tres semanas se procesan mediante un script Python desarrollado ad hoc que genera rankings de IPs atacantes con geolocalización, credenciales más utilizadas, comandos ejecutados y ficheros descargados. Los hallazgos se clasifican según el marco MITRE ATT&CK y se acompañan de recomendaciones defensivas aplicables a entornos reales. Como ampliación opcional se propone la ingesta de los logs en el SIEM Wazuh, no implementada en esta fase. El proyecto demuestra cómo un honeypot de bajo coste proporciona inteligencia de amenazas accionable con herramientas exclusivamente de software libre.

Palabras clave: honeypot, SSH, Cowrie, inteligencia de amenazas, MITRE ATT&CK, ciberseguridad, fuerza bruta, malware, SIEM, cloud

Abstract

This project covers the deployment and analysis of a medium-interaction SSH honeypot based on Cowrie, exposed to the Internet from a Hetzner Cloud server. The system simulates a vulnerable Linux server to attract automated attacks —primarily brute-force login attempts, command execution, and malware downloads— with the goal of studying the tactics, techniques, and procedures (TTPs) used by attackers. Data captured over a minimum three-week period is processed by a purpose-built Python script that generates attacker IP rankings with geolocation, most-used credentials, executed commands, and downloaded files. Findings are mapped to the MITRE ATT&CK framework and accompanied by defensive recommendations applicable to production environments. Ingesting the logs into the Wazuh SIEM is proposed as an optional extension and has not been implemented at this stage. The project demonstrates how a low-

cost honeypot provides actionable threat intelligence using exclusively open-source tools.

Keywords: honeypot, SSH, Cowrie, threat intelligence, MITRE ATT&CK, cybersecurity, brute force, malware, SIEM, cloud

Justificación del Proyecto

Motivación

La ciberseguridad se ha convertido en una disciplina esencial dentro de la administración de sistemas informáticos en red. Según el informe del Centro Criptológico Nacional (CCN-CERT), España registró más de 100.000 incidentes gestionados en 2024, con un crecimiento sostenido de los ataques automatizados contra servicios expuestos a Internet. Esta tendencia se acentúa en un contexto global donde la transformación digital ha incrementado exponencialmente la superficie de ataque de organizaciones de todos los tamaños.

El protocolo SSH (Secure Shell), presente en prácticamente todo servidor Linux y en muchos dispositivos de red, constituye uno de los vectores más atacados. Cualquier máquina con el puerto 22 abierto a Internet recibe intentos de fuerza bruta en cuestión de minutos tras su puesta en marcha. Estos ataques, ejecutados mayoritariamente por botnets, prueban combinaciones de credenciales comunes de forma masiva y automatizada, buscando servidores mal configurados que permitan el acceso con contraseñas débiles o por defecto.

Este proyecto nace de la voluntad de comprender de primera mano cómo operan estos ataques automatizados: qué credenciales prueban, con qué frecuencia y desde qué regiones geográficas; qué comandos ejecutan tras obtener acceso; y qué malware intentan desplegar en los sistemas comprometidos. Un honeypot permite observar todo esto en un entorno controlado y sin riesgo para infraestructura de producción, proporcionando datos reales que pueden utilizarse para mejorar las defensas de cualquier organización.

Desde una perspectiva formativa, el proyecto integra competencias de la mayoría de los módulos del ciclo, como se detalla en el apartado siguiente. Esta transversalidad lo convierte en un proyecto idóneo como Proyecto Intermodular.

Relación con los Módulos Profesionales

La Tabla 1 recoge los módulos del ciclo de ASIR que intervienen en el proyecto y, para cada uno, el trabajo concreto en el que se aplican sus contenidos. Se indica también el grado de relación, para distinguir los módulos en los que se apoya el núcleo del proyecto de aquellos con una aportación puntual.

Módulo	Aplicación en el proyecto	Relación
Seguridad y Alta Disponibilidad	Bastionado del SSH (puerto 2022, solo clave, sin root), cortafuegos en dos capas (Cloud Firewall e iptables), confinamiento del servicio con systemd, honeypot como técnica de detección y análisis de ataques con MITRE ATT&CK.	Principal
Administración de Sistemas Operativos	Gestión de usuarios y permisos (usuario administrador y usuario sin privilegios para Cowrie), servicios con systemd, scripts de administración en Bash, actualizaciones automáticas y diagnóstico con journalctl.	Principal
Servicios de Red e Internet	Configuración del servicio SSH (sshd_config, activación por socket en Ubuntu 24.04), autenticación por clave y transferencia de ficheros con scp.	Principal
Planificación y Administración de Redes	Direccionamiento IP público, redirección de puertos con NAT (PREROUTING), filtrado de tráfico y verificación con nmap.	Principal
Implantación de Sistemas Operativos	Despliegue de Ubuntu Server 24.04 en la nube, elección de imagen y arquitectura, entornos virtuales de Python y laboratorio en VirtualBox.	Alta
Lenguajes de Marcas y Sistemas de Gestión de Información	Procesamiento de los logs en formato JSON, exportación a CSV y diagramas del proyecto elaborados en HTML y SVG.	Media
Fundamentos de Hardware	Dimensionado del servidor (vCPU, memoria y disco) y diferencias entre arquitecturas x86 y ARM, que reaparecen en el malware capturado, compilado para cinco arquitecturas.	Media
Implantación de Aplicaciones Web	Consumo de la API REST de geolocalización y del servicio de notificaciones; publicación del proyecto en un repositorio Git.	Baja
Gestión de Bases de Datos / Administración de SGBD	Sin relación directa: los eventos se almacenan en ficheros JSON. La integración con el SIEM Wazuh, que indexa los eventos, se plantea como trabajo futuro.	Indirecta
Inglés Profesional	Documentación técnica en inglés (Cowrie, MITRE ATT&CK, informes de amenazas de Kaspersky y SANS) y redacción del Abstract.	Media
Digitalización Aplicada a los Sectores Productivos	Uso de servicios cloud, automatización del despliegue con scripts y del análisis con Python.	Media
Sostenibilidad Aplicada al Sistema Productivo	Dimensionado ajustado a la carga real y eliminación del servidor al terminar la captura, sin recursos ociosos.	Baja
Itinerario Personal para la Empleabilidad I y II	Proyecto orientado a perfiles de analista SOC y administración de sistemas, con repositorio público como portafolio profesional.	Media

Tabla 1. Relación del proyecto con los módulos profesionales del ciclo.

Estado de la Cuestión

Los honeypots son una herramienta consolidada en el ámbito de la ciberseguridad defensiva. El concepto fue formalizado por Lance Spitzner a principios de los años 2000, y desde entonces ha evolucionado significativamente en sofisticación y diversidad. Se clasifican habitualmente por su nivel de interacción:

- **Baja interacción:** simulan únicamente puertos abiertos y respuestas básicas de protocolo (p. ej., Honeyd, Dionaea). Capturan datos limitados —fundamentalmente IPs de origen y puertos escaneados— pero son triviales de desplegar y presentan un riesgo mínimo, ya que no ofrecen ningún acceso real al atacante.
- **Media interacción:** emulan un servicio completo, incluyendo autenticación, un entorno de comandos limitado y un filesystem simulado (p. ej., Cowrie, Kippo). Ofrecen un equilibrio óptimo entre la cantidad de datos capturados y el riesgo asumido, permitiendo observar el comportamiento del atacante después de la autenticación sin exponer un sistema real.
- **Alta interacción:** proporcionan un sistema operativo real al atacante, generalmente mediante máquinas virtuales o contenedores aislados (p. ej., HonSSH combinado con VM, Cowrie con proxy a backend real). Capturan la mayor cantidad de datos posible, incluyendo binarios compilados por el atacante y técnicas de post-explotación completas, pero requieren un aislamiento riguroso y una monitorización constante para evitar que el sistema comprometido se utilice como plataforma de ataque.

En la literatura académica, estudios como los de Provos y Holz (2007) en «Virtual Honeypots» y los informes periódicos del proyecto Honeynet demuestran el valor de los honeypots como fuente de inteligencia de amenazas. Investigaciones recientes (2023-2024) publican análisis similares al propuesto en este proyecto, confirmando que los patrones de ataque SSH se mantienen notablemente estables: las mismas credenciales triviales (root/123456, admin/admin) y los mismos comandos de reconocimiento se observan año tras año, lo que sugiere

que una proporción significativa de los ataques los realizan botnets con scripts automatizados y poco sofisticados.

Cowrie es el honeypot SSH/Telnet de media interacción más utilizado en la comunidad de ciberseguridad. Desarrollado en Python y mantenido activamente en GitHub con una comunidad de contribuidores activa, permite registrar credenciales probadas, comandos ejecutados, ficheros descargados e incluso grabar sesiones TTY completas. Su formato de salida JSON facilita el procesamiento automatizado y la integración con SIEMs como Wazuh, ELK o Splunk. Cowrie hereda la base de Kippo pero supera ampliamente sus capacidades, incluyendo soporte para sftp, ejecución en Docker y un sistema extensible de plugins de salida.

Comparativa Razonada

Para seleccionar la herramienta central del proyecto se evaluaron cuatro alternativas, considerando criterios de funcionalidad, requisitos de recursos y adecuación al entorno disponible:

Criterio	Cowrie	Kippo	Honeyd	T-Pot
Nivel de interacción	Media	Media	Baja	Multi
Mantenimiento activo	Sí	No (abandonado)	Limitado	Sí
RAM mínima	512 MB	256 MB	128 MB	8 GB
Captura comandos	Sí	Sí	No	Sí
Salida JSON	Sí	No	No	Sí
Adecuación a servidor pequeño	Óptima	Aceptable	Limitada	Inviabile

Tabla 2. Comparativa de honeypots SSH.

Cowrie ofrece la mejor relación entre profundidad de captura, facilidad de instalación y requisitos hardware. Esto lo hace especialmente adecuado para un proyecto académico desplegado en un servidor cloud pequeño, con pocos núcleos y unos pocos GB de RAM. Kippo, su predecesor, está abandonado y no recibe actualizaciones de seguridad. Honeyd solo ofrece baja interacción, insuficiente para observar el comportamiento post-autenticación. T-Pot integra múltiples honeypots pero requiere 8 GB de RAM como mínimo, lo que lo hace inviable en un servidor de ese tamaño.

La elección del marco MITRE ATT&CK para clasificar la actividad observada responde a su adopción generalizada en la industria como lenguaje común para describir comportamientos adversarios. Desarrollado y mantenido por MITRE Corporation con actualizaciones trimestrales, el framework cataloga las tácticas (el «por qué» de una acción), las técnicas (el «cómo») y los procedimientos (la implementación específica). Permite traducir los hallazgos técnicos del honeypot a un formato que cualquier equipo de seguridad puede interpretar y utilizar para mejorar sus defensas, facilitando además la comparación de hallazgos entre diferentes investigaciones y organizaciones.

Introducción

El presente proyecto consiste en el despliegue de un honeypot SSH en la nube pública, la captura de ataques reales durante un período prolongado y el análisis de los datos obtenidos para generar un informe de inteligencia de amenazas. Se enmarca en el Proyecto Intermodular del ciclo de ASIR, integrando competencias de varias asignaturas: administración de sistemas Linux, seguridad informática, scripting y redes.

El sistema se compone de tres elementos principales: un servidor cloud con el honeypot Cowrie expuesto a Internet, un conjunto de scripts Python para el análisis automatizado de los logs y un laboratorio local donde se procesan los datos y se genera el informe final. El resultado es un flujo completo de inteligencia de amenazas, desde la captura de datos brutos hasta la producción de artefactos accionables como rankings de IPs, gráficas estadísticas y un mapeo MITRE ATT&CK con recomendaciones defensivas.

Este documento constituye la memoria técnica del proyecto, describiendo la arquitectura del sistema, las decisiones de diseño, la metodología seguida, las tecnologías empleadas y las conclusiones extraídas del análisis de los ataques capturados.

Evolución respecto al Anteproyecto

El anteproyecto planteaba la idea y un desarrollo en seis fases. Durante esta primera etapa el proyecto se ha estructurado en ocho fases con el desglose RFTP, se han automatizado el despliegue y el análisis, y el honeypot está en funcionamiento desde el 4 de octubre de 2026. La Tabla 3 resume los cambios y su motivo.

Aspecto	Anteproyecto	Situación actual y motivo
Plataforma cloud	Oracle Cloud, capa gratuita	Hetzner Cloud (CX23, Núremberg). Oracle no tuvo capacidad en más de 600 solicitudes y no había otras alternativas gratuitas viables.
Cortafuegos del servidor	UFW	iptables, más el Cloud Firewall del proveedor. UFW no permite crear la redirección NAT del puerto 22 con sus comandos.
Despliegue	Pasos manuales	Tres scripts idempotentes con comprobaciones que impiden perder el acceso al servidor.
Análisis	Scripts Python para rankings y gráficas	Analizador modular con geolocalización, ficheros capturados por hash, clientes SSH y clasificación automática en MITRE ATT&CK con 50 pruebas.
Planificación	Seis fases	Ocho fases con desglose RFTP, diagrama de Gantt y estimación de 109 horas.

Aspecto	Anteproyecto	Situación actual y motivo
Captura	Dos o tres semanas	Iniciada el 4 de octubre, diez días antes de lo previsto, y ampliada hasta finales de octubre.
Coste	Gratuito	Unos 15 € en total por el cambio de proveedor (véase el Presupuesto).

Tabla 3. Evolución del proyecto respecto al anteproyecto.

Funciones Principales

El sistema desarrollado cumple las siguientes funciones:

- Simular un servidor SSH vulnerable accesible desde Internet para atraer ataques automatizados de forma segura, sin exponer infraestructura de producción.
- Registrar de forma detallada cada intento de acceso: IPs de origen, credenciales probadas, timestamps, comandos ejecutados tras el acceso y ficheros descargados.
- Analizar los datos capturados de forma automatizada mediante un script Python que genera estadísticas, rankings y gráficas exportables en formato PNG.
- Geolocalizar las IPs atacantes para determinar la distribución geográfica de las fuentes de ataque.
- Clasificar la actividad observada según el marco MITRE ATT&CK, proporcionando un informe de inteligencia accionable con recomendaciones defensivas concretas.
- Opcionalmente, integrar los logs con un SIEM (Wazuh) para su visualización en dashboard y la configuración de alertas en tiempo real.

Requisitos del Sistema

Para la ejecución completa del proyecto se requiere:

Requisitos Hardware

- Servidor cloud con al menos 1 vCPU y 1 GB de RAM (en este proyecto se utiliza un servidor CX23 de Hetzner Cloud con 2 vCPU x86 y 4 GB de RAM).
- Dirección IP pública accesible desde Internet.
- Equipo local con capacidad de virtualización para ejecutar Wazuh y Kali Linux (mínimo 8 GB de RAM total en el host).

Requisitos Software

- Cuenta en un proveedor cloud con servidores Linux e IP pública (en este proyecto, Hetzner Cloud).
- Ubuntu Server 24.04 LTS como sistema operativo del servidor cloud.
- OpenSSH configurado con autenticación por clave pública ed25519.
- Cowrie 3.1.0, instalado desde PyPI.
- Python 3.10 o superior con las bibliotecas json, collections, matplotlib y requests.
- VirtualBox 7.x como hipervisor local para las máquinas virtuales de laboratorio.
- Wazuh 4.9.2 en máquina virtual local (componente opcional; su integración con el honeypot no se ha implementado en esta fase).
- Kali Linux (última versión rolling) para pruebas de ataque controlado.
- Git para el control de versiones del proyecto.

Requisitos de Red

- Conexión a Internet permanente en el servidor cloud para la exposición del honeypot.
- Puertos 22 (honeypot) y 2022 (SSH real) accesibles desde Internet a través del firewall del proveedor (Cloud Firewall de Hetzner).
- Acceso a la API de ip-api.com para la geolocalización de IPs atacantes.
- Conectividad SSH (puerto 2022) entre el laboratorio local y el servidor cloud para la descarga de logs.

Objetivos

Objetivo General

Desplegar y operar un honeypot SSH expuesto a Internet que capture ataques reales durante al menos tres semanas, y transformar esos datos en un informe de inteligencia de amenazas con la actividad clasificada según MITRE ATT&CK y recomendaciones defensivas aplicables a cualquier servidor Linux.

Objetivos Específicos

Cada objetivo específico se acompaña de un indicador verificable y de su estado a fecha de esta entrega (8 de octubre de 2026). Un objetivo «en curso» puede tener ya resultados parciales: es el caso de la captura (OE4), cuyos tres primeros días se analizan en el capítulo de Resultados con carácter provisional, aunque el objetivo solo se dará por cumplido al completar las tres semanas.

Objetivo específico	Indicador de cumplimiento	Estado
OE1. Desplegar un servidor cloud bastionado cuyo acceso de administración no pueda comprometerse por fuerza bruta.	Escaneo externo con solo los puertos 22 y 2022 visibles; acceso como root y con contraseña rechazados en el 2022.	Cumplido
OE2. Instalar Cowrie con una identidad verosímil y registrar toda la actividad.	nmap identifica el puerto 22 como OpenSSH de Debian 12; los atacantes ejecutan comandos; cada evento queda en cowrie.json.	Cumplido
OE3. Automatizar el despliegue para que sea reproducible en cualquier proveedor.	Un Ubuntu 24.04 recién creado queda operativo ejecutando tres scripts, sin pasos manuales adicionales.	Cumplido
OE4. Capturar ataques reales durante un periodo suficiente.	Al menos tres semanas de captura continua.	En curso (desde el 4 de octubre); resultados provisionales de 3 días
OE5. Desarrollar un analizador automático de los logs.	Rankings, geolocalización, indicadores de compromiso y gráficas generados en una sola ejecución sobre más de 10 000 sesiones.	Cumplido
OE6. Clasificar automáticamente la actividad en MITRE ATT&CK.	Al menos el 95 % de los comandos con técnica asignada y un conjunto de pruebas automáticas superado.	Cumplido sobre los datos parciales (98,5

Objetivo específico	Indicador de cumplimiento	Estado
		%; 50/50 pruebas); se verificará con el periodo completo
OE7. Elaborar el informe de inteligencia de amenazas.	Informe con campañas identificadas, indicadores de compromiso y recomendaciones vinculadas a mitigaciones de ATT&CK.	En curso
OE8. Documentar el proyecto de forma reproducible.	Memoria con el formato de Prometeo y repositorio público con scripts, configuración y capturas.	En curso

Tabla 4. Objetivos específicos, indicadores y estado.

Desglose RFTP

A continuación se presenta el desglose completo del proyecto siguiendo la estructura RFTP (Requisitos, Funciones, Tareas, Pruebas) establecida por la plantilla de Prometeo FP. Cada requisito se descompone jerárquicamente con la codificación R > F > T > P.

Código	Descripción	Tipo	Prueba
R01	Despliegue de infraestructura cloud	REQ	
R01F01	Provisionar servidor en Hetzner Cloud	FUNC	
R01F01T01	Crear cuenta y proyecto en Hetzner Cloud con clave SSH dedicada	TAREA	
R01F01T01P01	Acceso a la consola con 2FA y clave SSH registrada	PRUEBA	✓
R01F01T02	Crear el Cloud Firewall (entrada solo TCP 22, TCP 2022 e ICMP)	TAREA	
R01F01T02P01	Ping externo a la IP pública del servidor	PRUEBA	✓
R01F01T03	Desplegar servidor CX23 (x86, Ubuntu 24.04, Núremberg)	TAREA	
R01F01T03P01	Conexión SSH al servidor desde terminal local	PRUEBA	✓
R01F02	Securizar el servidor (hardening)	FUNC	
R01F02T01	Configurar firewall iptables persistente: abrir 2022, rechazar el resto	TAREA	
R01F02T01P01	nmap muestra solo puertos 22 y 2022 abiertos	PRUEBA	✓
R01F02T02	Mover SSH real al puerto 2022 con clave ed25519	TAREA	
R01F02T02P01	Conexión SSH al puerto 2022 con clave, rechazo con contraseña	PRUEBA	✓
R01F02T	Deshabilitar login root y autenticación por contraseña	TAREA	

Código	Descripción	Tipo	Prueba
03		A	
R01F02T 03P01	Intento de login root rechazado	PRUEBA	✓
R02	Instalación y configuración del honeypot	REQ	
R02F01	Instalar y configurar Cowrie	FUNC	
R02F01T 01	Instalar Cowrie en virtualenv con dependencias	TAREA	
R02F01T 01P01	Cowrie arranca sin errores y escucha en :2222	PRUEBA	✓
R02F01T 02	Configurar cowrie.cfg (hostname, filesystem, credenciales)	TAREA	
R02F01T 02P01	Conexión SSH al honeypot muestra prompt falso	PRUEBA	✓
R02F01T 03	Configurar iptables para redirigir :22 → :2222	TAREA	
R02F01T 03P01	Conexión a puerto 22 aterriza en Cowrie	PRUEBA	✓
R02F01T 04	Configurar output en formato JSON	TAREA	
R02F01T 04P01	cowrie.json se genera con eventos estructurados	PRUEBA	✓
R03	Captura de datos de ataques reales	REQ	
R03F01	Recopilar ataques durante período de exposición	FUNC	
R03F01T 01	Mantener Cowrie expuesto a Internet durante ≥3 semanas	TAREA	
R03F01T 01P01	cowrie.json contiene ≥1000 eventos de distintas IPs	PRUEBA	✓
R03F01T 02	Monitorizar servicio periódicamente y reiniciar si cae	TAREA	
R03F01T 02P01	Logs continuos sin gaps mayores a 24h	PRUEBA	✓
R03F01T 03	Descargar los logs al laboratorio local por SSH (puerto 2022)	TAREA	
R03F01T 03P01	Fichero cowrie.json íntegro en máquina local	PRUEBA	✓
R04	Análisis automatizado con Python	REQ	
R04F01	Desarrollar script de análisis (cowrie_analyzer.py)	FUNC	
R04F01T 01	Parsear logs JSON y extraer métricas clave	TAREA	
R04F01T 01P01	Script procesa sample_cowrie.json sin errores	PRUEBA	✓
R04F01T 02	Ranking de IPs atacantes con geolocalización	TAREA	
R04F01T 02P01	Top 10 IPs mostradas con país y ciudad	PRUEBA	✓
R04F01T 03	Ranking de credenciales (usuario/contraseña)	TAREA	

Código	Descripción	Tipo	Prueba
R04F01T03P01	Gráfica top_passwords.png generada correctamente	PRUEBA	✓
R04F01T04	Análisis de comandos ejecutados tras acceso	TAREA	
R04F01T04P01	Lista de comandos con frecuencias	PRUEBA	✓
R04F01T05	Detección de descargas de malware	TAREA	
R04F01T05P01	URLs y hashes SHA de ficheros descargados listados	PRUEBA	✓
R04F01T06	Generar gráficas con matplotlib	TAREA	
R04F01T06P01	Gráficas PNG en directorio de salida	PRUEBA	✓
R04F02	Generar datos de prueba	FUNC	
R04F02T01	Script generate_sample_logs.py con 500 sesiones simuladas	TAREA	
R04F02T01P01	sample_cowrie.json generado con distribución realista	PRUEBA	✓
R05	Informe de inteligencia de amenazas	REQ	
R05F01	Elaborar informe CTI con mapeo MITRE ATT&CK	FUNC	
R05F01T01	Clasificar actividad observada con MITRE ATT&CK	TAREA	
R05F01T01P01	Cada técnica mapeada con ID y evidencia del log	PRUEBA	✓
R05F01T02	Redactar informe de resumen ejecutivo	TAREA	
R05F01T02P01	Informe revisado y coherente	PRUEBA	✓
R05F01T03	Incluir recomendaciones de defensa	TAREA	
R05F01T03P01	Cada técnica acompañada de mitigación	PRUEBA	✓
R06	Integración con SIEM (Wazuh) — opcional, no implementada	REQ	
R06F01	Ingesta de logs en Wazuh (opcional)	FUNC	
R06F01T01	Instalar Wazuh 4.9.2 en VM local	TAREA	
R06F01T01P01	Dashboard Wazuh accesible en navegador	PRUEBA	✓
R06F01T02	Configurar ingesta de cowrie.json	TAREA	
R06F01T02P01	Eventos de Cowrie visibles en Wazuh Discover	PRUEBA	✓
R07	Documentación y entrega	REQ	
R07F01	Redactar memoria del proyecto	FUNC	
R07F01T	Redactar memoria siguiendo plantilla Prometeo	TAREA	

Código	Descripción	Tipo	Prueba
01		A	
R07F01T 01P01	Documento cumple estructura y formato APA	PRUEBA	✓
R07F01T 02	Incluir capturas, gráficas y diagramas	TAREA	
R07F01T 02P01	Todas las figuras referenciadas y visibles	PRUEBA	✓
R07F02	Gestionar repositorio GitHub	FUNC	
R07F02T 01	Mantener repositorio PFC-Cowrie con README y commits	TAREA	
R07F02T 01P01	Repositorio público con historial de commits	PRUEBA	✓

Descripción del Sistema

Estado de los Componentes

Para evitar confusiones entre lo construido y lo planteado, la Tabla 5 indica el estado de cada componente del sistema a fecha de esta entrega (8 de octubre de 2026). Los resultados que se presentan en el capítulo de Resultados son provisionales: proceden de los tres primeros días de una captura que continúa hasta finales de octubre.

Componente	Estado	Evidencia
Servidor en Hetzner Cloud (x86_64) y Cloud Firewall	Implementado y probado	Escaneo nmap externo (capítulo de Pruebas)
Bastionado del SSH de administración	Implementado y probado	Acceso como root rechazado; reglas persistentes tras reinicio
Cowrie como servicio y redirección del puerto 22	Implementado y probado	Pruebas local y externa; ataques reales registrados
Captura de ataques reales	En curso	Desde el 4 de octubre; objetivo de al menos tres semanas
Analizador y clasificador MITRE ATT&CK	Implementado y probado	50 pruebas automáticas; análisis de 13 761 sesiones reales
Resultados y conclusiones	Provisionales	Basados en los tres primeros días de captura
Informe de inteligencia de amenazas	En elaboración	Se completará con el periodo de captura completo
Ataque controlado desde Kali Linux	Pendiente	Última prueba de la tabla RFTP (CU-03)
Integración con Wazuh (SIEM)	Propuesto, no implementado	Requisito opcional R06; trabajo futuro

Tabla 5. Estado de implementación de los componentes.

Arquitectura General

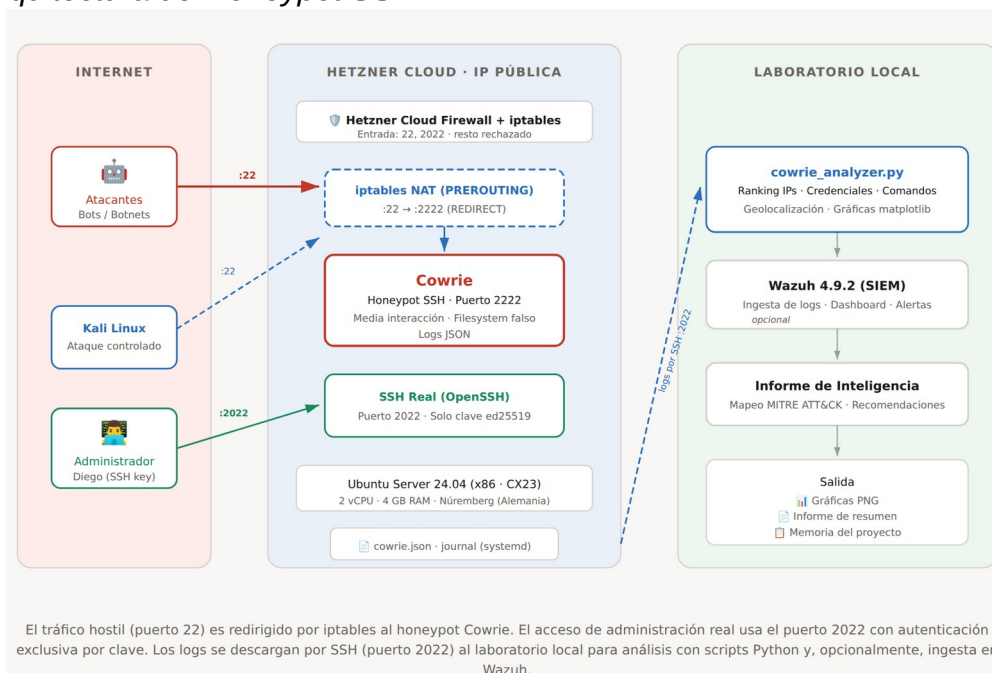
La arquitectura del sistema se organiza en tres zonas diferenciadas, como muestra la Figura 1:

- **Zona Internet:** los atacantes (bots y botnets) conectan al puerto 22 de la IP pública. Desde esta zona también opera Kali Linux para pruebas controladas. El administrador accede por el puerto 2022.
- **Zona Hetzner Cloud:** el servidor Ubuntu 24.04 (CX23, 2 vCPU x86, 4 GB RAM, en Núremberg) ejecuta iptables como firewall del host y para la redirección NAT del puerto 22 al 2222, Cowrie como honeypot y OpenSSH en el puerto 2022 para administración real.
- **Zona Laboratorio Local:** los logs se descargan por SSH (puerto 2022) y se analizan con el script Python (cowrie_analyzer.py). La

ingesta en Wazuh (SIEM) se plantea como ampliación opcional, no implementada. Se genera el informe de inteligencia con mapeo MITRE ATT&CK.

Figura 1

Arquitectura del honeypot SSH



Flujo de Datos

El flujo de un ataque típico sigue estos pasos:

- El atacante (bot) descubre la IP pública mediante escaneo masivo de Internet.
- Envía una conexión SSH al puerto 22 de la IP pública del servidor.
- El Cloud Firewall de Hetzner admite el tráfico al puerto 22. Ya en el servidor, la regla NAT PREROUTING de iptables redirige la conexión al puerto 2222, donde escucha Cowrie.
- Cowrie presenta un banner SSH y acepta intentos de autenticación. Cada intento (exitoso o fallido) se registra en cowrie.json con la IP de origen, usuario, contraseña y timestamp.
- Si Cowrie permite el acceso (configurable), el atacante interactúa con un filesystem falso. Los comandos ejecutados se registran como eventos cowrie.command.input.

- Si el atacante intenta descargar malware (wget, curl), Cowrie registra la URL y el hash SHA del fichero como evento `cowrie.session.file_download`.
- Al terminar, se registra `cowrie.session.closed` con la duración de la sesión.
- Los logs acumulados se transfieren periódicamente al laboratorio local para su análisis.

Seguridad y Aislamiento

Un aspecto crítico del diseño es la separación estricta entre el tráfico hostil (dirigido al honeypot) y el tráfico legítimo de administración. Esta separación se consigue mediante tres niveles de control:

- **Nivel 1 — Infraestructura cloud (Cloud Firewall):** el Cloud Firewall de Hetzner (honeypot-fw) permite exclusivamente el tráfico entrante TCP a los puertos 22 y 2022 e ICMP. Cualquier otro puerto se bloquea en la infraestructura del proveedor, antes de llegar al servidor.
- **Nivel 2 — Firewall del host (iptables):** como segunda capa, la cadena INPUT de iptables rechaza todo el tráfico entrante salvo las conexiones ya establecidas, el SSH real (2022) y el puerto interno de Cowrie (2222). Las reglas se guardan con `netfilter-persistent` para sobrevivir a los reinicios.
- **Nivel 3 — Redirección NAT (iptables):** la regla PREROUTING redirige todo el tráfico al puerto 22 hacia el puerto 2222 (Cowrie), asegurando que un atacante que conecta al puerto estándar de SSH interactúa exclusivamente con el honeypot. El SSH real en el puerto 2022 no se ve afectado por esta regla.

Esta defensa en profundidad garantiza que un fallo en cualquiera de los niveles individuales no compromete la seguridad del SSH de administración. Además, la autenticación del SSH real se restringe exclusivamente a claves `ed25519`, eliminando por completo la posibilidad de un acceso por fuerza bruta al servicio legítimo.

Casos de Uso

Caso de uso	CU-01: Captura de ataque SSH
Actores	Atacante (bot), Honeypot (Cowrie)
Descripción	Un atacante remoto intenta acceder al servidor SSH expuesto. Cowrie registra todos los eventos de la sesión.
Precondiciones	Cowrie está en ejecución y escuchando en el puerto 2222.

	iptables redirige :22 → :2222.
Postcondiciones	Los eventos de la sesión quedan registrados en cowrie.json con todos los campos (IP, credenciales, comandos, descargas).
Datos de entrada	Conexión SSH con credenciales de fuerza bruta
Datos de salida	Eventos JSON estructurados en cowrie.json

Caso de uso	CU-02: Análisis de logs con Python
Actores	Administrador (Diego)
Descripción	El administrador ejecuta cowrie_analyzer.py sobre el fichero de logs para obtener métricas, gráficas y un informe de resumen.
Precondiciones	Fichero cowrie.json disponible en el laboratorio local. Python 3 con matplotlib instalado.
Postcondiciones	Gráficas PNG generadas, informe de resumen en texto, datos listos para mapeo MITRE.
Datos de entrada	cowrie.json (logs del honeypot)
Datos de salida	Gráficas PNG, informe .txt, ranking IPs con geolocalización

Caso de uso	CU-03: Ataque controlado desde Kali Linux
Actores	Administrador (Diego), Kali Linux, Honeypot (Cowrie)
Descripción	El administrador ejecuta un ataque de fuerza bruta controlado desde Kali Linux para verificar el correcto funcionamiento del honeypot y la generación de logs.
Precondiciones	Cowrie operativo. Kali Linux con herramientas hydra y nmap disponibles. Conectividad SSH al servidor cloud.
Postcondiciones	Los logs reflejan los intentos de fuerza bruta y los comandos ejecutados durante la prueba controlada.
Datos de entrada	Diccionario de credenciales (hydra), comandos de prueba manuales
Datos de salida	Eventos JSON en cowrie.json correspondientes a la sesión de prueba

Diseños

Diagrama de Arquitectura de Red

La Figura 1 (incluida en la sección de Descripción) muestra la arquitectura completa del sistema. A continuación se detalla el diseño de red a nivel lógico, incluyendo las tres zonas de seguridad, las interfaces de red y las reglas de filtrado que rigen el tráfico entre ellas.

Zona Internet (no confiable)

Esta zona incluye todo el tráfico procedente de Internet. Se subdivide en dos tipos de actores:

- **Atacantes (tráfico hostil):** bots y botnets que escanean rangos de IPs en busca del puerto 22 abierto. Su tráfico se redirige al honeypot Cowrie mediante iptables NAT. Nunca tienen acceso al SSH real ni a otros servicios del servidor.
- **Administrador (tráfico legítimo):** acceso SSH al puerto 2022 con autenticación por clave ed25519. Este es el único camino de administración real del servidor. El acceso se restringe por clave pública; la autenticación por contraseña está deshabilitada.
- **Kali Linux (pruebas controladas):** accede al puerto 22 para verificar que Cowrie responde correctamente y que los logs se generan según lo esperado. Su tráfico es indistinguible del de un atacante real desde la perspectiva del honeypot.

Zona DMZ Cloud (Hetzner Cloud)

El servidor cloud actúa como una DMZ funcional que aísla el honeypot del tráfico legítimo. La separación se consigue mediante la combinación de tres mecanismos:

- **Cloud Firewall de Hetzner:** reglas de firewall a nivel de la infraestructura cloud que permiten únicamente el tráfico entrante TCP a los puertos 22 y 2022 e ICMP. Todo el demás tráfico se descarta antes de llegar al servidor.
- **iptables (firewall del host):** segunda capa de filtrado en el sistema operativo que rechaza todo lo no autorizado, proporcionando defensa en profundidad. En caso de error en el Cloud Firewall, iptables actúa como respaldo.

- **iptables NAT:** regla PREROUTING que redirige el tráfico destinado al puerto 22 hacia el puerto 2222 (Cowrie), asegurando que ningún atacante interactúe con el SSH real.

Zona Laboratorio Local (confiable)

El laboratorio local es una red aislada compuesta por máquinas virtuales VirtualBox. No está expuesta a Internet; la comunicación con el servidor cloud se establece bajo demanda mediante SSH para la descarga de logs. Este aislamiento garantiza que el análisis de datos potencialmente maliciosos (logs con URLs de malware, hashes de ficheros descargados) se realiza en un entorno seguro.

Diseño del Flujo de Datos

El sistema procesa los datos en tres etapas claramente diferenciadas, cada una con su propio formato de entrada y salida:

Etapas 1: Captura (Cowrie)

Cowrie registra cada evento como un objeto JSON independiente en el fichero cowrie.json. Los tipos de evento principales son:

- **cowrie.session.connect:** nueva conexión SSH. Campos: timestamp, src_ip, src_port, dst_ip, dst_port, session.
- **cowrie.login.failed / cowrie.login.success:** intento de autenticación. Campos: username, password, session.
- **cowrie.command.input:** comando ejecutado por el atacante. Campos: input, session.
- **cowrie.session.file_download:** descarga de fichero. Campos: url, shasum, outfile, session.
- **cowrie.session.closed:** fin de sesión. Campos: duration, session.

Etapas 2: Análisis (cowrie_analyzer.py)

El script Python procesa el fichero JSON línea a línea, contabilizando las métricas clave con la biblioteca collections.Counter. Para cada IP atacante, realiza una consulta a la API de ip-api.com para obtener país, ciudad e ISP. Las métricas se exportan como gráficas PNG mediante matplotlib y como un informe de resumen en texto plano.

Etapas 3: Inteligencia (Informe CTI)

Los hallazgos del análisis se mapean al marco MITRE ATT&CK, asignando a cada comportamiento observado su técnica correspondiente. El informe final incluye la evidencia del log (evento concreto), la técnica MITRE (ID y nombre) y una recomendación defensiva. Este formato permite que un equipo de seguridad comprenda rápidamente qué se observó, qué significa y cómo protegerse.

Diseño de Interfaces

El proyecto no dispone de una interfaz gráfica de usuario propia, al tratarse de un sistema basado en línea de comandos y ficheros de datos. Las interfaces relevantes son las siguientes:

Interfaz CLI del Script de Análisis

El script `cowrie_analyzer.py` se ejecuta desde terminal con la siguiente sintaxis:

```
python3 cowrie_analyzer.py <fichero.json> --output <directorio/> [--no-geo]
```

Los parámetros son:

- **<fichero.json>**: ruta al fichero de logs de Cowrie (obligatorio).
- **--output <directorio/>**: directorio donde se guardan las gráficas PNG y el informe de resumen (obligatorio).
- **--no-geo**: omite la geolocalización de IPs (útil para ejecución sin conexión a Internet).

La salida del script se muestra en terminal con formato de texto plano, incluyendo tablas con los rankings de IPs, credenciales y comandos. Las gráficas se generan como ficheros PNG independientes.

Interfaz del Honeypot (Vista del Atacante)

Lo que el atacante ve al conectarse al puerto 22 es un banner SSH estándar que identifica al servidor como un OpenSSH 9.2p1 de Debian 12, con el nombre de máquina `web-prod-02`. Tras autenticarse con credenciales aceptadas por Cowrie, accede a un prompt bash simulado con un filesystem falso que incluye directorios habituales (`/etc`, `/home`, `/var`) y ficheros falsos como `/etc/passwd`. Los comandos básicos (`ls`, `cat`,

wget, uname) están emulados; los no reconocidos producen un error «command not found» verosímil.

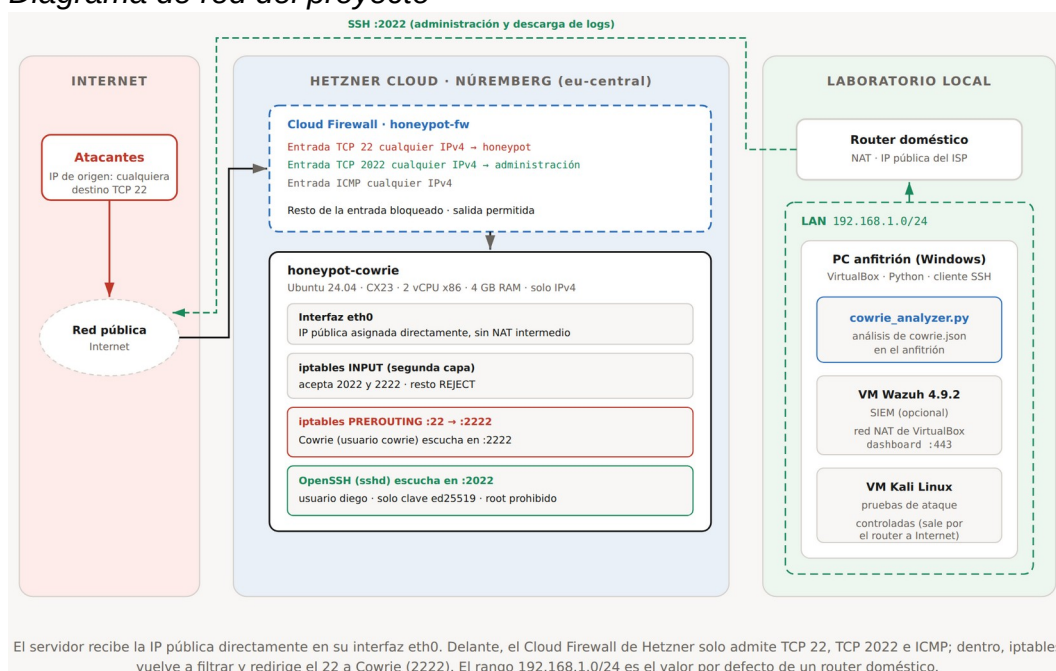
Dashboard de Wazuh (Propuesta, no Implementada)

Este apartado describe un diseño propuesto, no una interfaz construida: en esta fase los logs no se han integrado en Wazuh. Wazuh proporciona un dashboard web (puerto 443 de la VM local) con capacidades de búsqueda, filtrado y visualización de los eventos ingeridos. Si se realiza la integración, los logs de Cowrie se mostrarían en la sección Discover, donde pueden filtrarse por tipo de evento, IP de origen, rango temporal y otros campos. Se pueden crear visualizaciones personalizadas (gráficas de barras, mapas, tablas) agrupando los eventos por cualquier campo del JSON original.

Diagrama de Red

Figura 3

Diagrama de red del proyecto



El diagrama de red muestra la topología a nivel de direccionamiento IP:

- **Servidor Cloud:** Hetzner asigna la IP pública directamente a la interfaz eth0 del servidor, sin red privada ni traducción de direcciones intermedia. Solo se ha habilitado IPv4: el servidor se

creó sin IPv6 porque los scripts de bastionado configuran únicamente el filtrado IPv4.

- **Cloud Firewall:** cortafuegos gestionado por Hetzner que se aplica al tráfico antes de que llegue al servidor. Es la primera de las dos capas de filtrado del diseño.
- **Laboratorio Local:** red interna 192.168.1.0/24. Las máquinas virtuales (Wazuh, Kali) se conectan en modo NAT o bridge según la necesidad. La comunicación con el servidor cloud se realiza por Internet mediante SSH al puerto 2022. El fichero de logs pertenece al usuario sin privilegios cowrie, por lo que se lee con sudo en el servidor y se vuelca por la propia conexión SSH.

Tecnología

A continuación se describen las tecnologías y herramientas utilizadas en el proyecto.

Cowrie

Honeypot SSH/Telnet de media interacción, desarrollado en Python y mantenido como proyecto de código abierto en GitHub. Simula un sistema Linux completo con filesystem falso, permite configurar qué credenciales aceptar y registra toda la actividad en formato JSON. Soporta grabación de sesiones TTY y descarga segura de ficheros que los atacantes intentan transferir. En este proyecto, Cowrie constituye el componente central de captura de datos.

Hetzner Cloud

Proveedor cloud europeo en el que se ha desplegado el honeypot. Se utiliza un servidor CX23 (2 vCPU x86 compartidas, 4 GB de RAM y 40 GB de disco) en el centro de datos de Núremberg (Alemania), con Ubuntu Server 24.04 LTS y una dirección IPv4 pública asignada directamente a su interfaz de red. Delante del servidor se configura el Cloud Firewall de Hetzner, que solo admite tráfico entrante TCP a los puertos 22 y 2022 e ICMP. El acceso a la consola está protegido con autenticación en dos pasos, y el servidor se creó con una clave SSH dedicada exclusivamente al proyecto.

Plan Inicial y Cambio de Proveedor

El anteproyecto preveía desplegar el servidor en el tier gratuito de Oracle Cloud Infrastructure (OCI), que ofrece instancias ARM (Ampere A1.Flex) de hasta 4 OCPU y 24 GB de RAM sin coste. Sin embargo, estas instancias son muy demandadas y la región de Madrid respondía de forma sistemática con el error «Out of host capacity».

Para intentar obtenerla se desarrolló el script `oci_a1_retry.py`, que, con el SDK oficial de Oracle para Python, solicitaba la instancia cada minuto (véase el capítulo de Implementación). Tras más de 600 intentos a lo largo de más de un día sin éxito, se valoraron las alternativas gratuitas: la otra instancia gratuita de Oracle (VM.Standard.E2.1.Micro), que no aparecía disponible para la cuenta en la región; el paso a una cuenta de pago de Oracle, que exigía una retención temporal en la tarjeta; Azure for

Students, cuya verificación automática no reconoce el dominio de correo del centro; y el nuevo plan gratuito con crédito de AWS, al que no tenía derecho una cuenta creada antes de julio de 2025.

Agotadas las opciones gratuitas, se eligió Hetzner Cloud por tres motivos: el precio cerrado y bajo de sus servidores pequeños, la facturación por horas sin permanencia (el servidor se elimina al terminar el proyecto) y la disponibilidad inmediata. Gracias a que todo el despliegue está automatizado con scripts genéricos para Ubuntu 24.04, el cambio de proveedor solo afectó a la creación de la máquina; los scripts únicamente se adaptaron para que los mensajes no mencionaran Oracle y para crear el usuario administrador cuando el proveedor solo entrega acceso como root, como ocurre en Hetzner.

Recuperación de Instancias Ociosas

Mientras Oracle era la plataforma prevista, se analizó además su política de recuperación de instancias ociosas. Oracle se reserva el derecho de recuperar las instancias Always Free que considera ociosas: aquellas que durante 7 días mantienen a la vez un uso de CPU inferior al 20 % (percentil 95), un uso de red inferior al 20 % y, en las instancias A1, un uso de memoria inferior al 20 % (Oracle, s. f.-a). Un honeypot de media interacción cumple las tres condiciones, ya que Cowrie consume muy pocos recursos, por lo que la instancia corría el riesgo de ser recuperada en mitad del periodo de captura.

Dado que las tres condiciones deben cumplirse simultáneamente, basta con evitar una de ellas. Se desarrolló un pequeño servicio, oci-keepalive, que reserva el 25 % de la memoria RAM y la mantiene ocupada sin consumir CPU. El servicio se ejecuta con un usuario efímero sin privilegios, sin acceso a la red ni al sistema de ficheros, y con el uso de swap deshabilitado para garantizar que la reserva permanezca en memoria física. Se descartó la alternativa de generar carga de CPU artificial por ser un consumo innecesario de recursos compartidos. En el despliegue final en Hetzner esta política no existe, y el script de instalación no activa el servicio: lo reserva para instancias A1 de Oracle, que detecta por la etiqueta de chasis OracleCloud.com del sistema.

Python 3

Lenguaje de scripting utilizado para el desarrollo del analizador de logs (cowrie_analyzer.py) y el generador de datos de prueba

(generate_sample_logs.py). Se emplean las bibliotecas estándar json y collections para el procesamiento de datos, matplotlib para la generación de gráficas, y requests para la geolocalización de IPs mediante la API de ip-api.com.

Wazuh 4.9.2

Plataforma SIEM (Security Information and Event Management) de código abierto, derivada de OSSEC. Está disponible en el laboratorio local de virtualización del autor, pero su integración con el honeypot (ingesta de cowrie.json y reglas de alerta) no se ha realizado en esta fase: se mantiene como componente opcional del requisito R06 y como línea de trabajo futuro. El análisis del proyecto no depende de él, ya que lo realiza el script cowrie_analyzer.py.

MITRE ATT&CK

Marco de referencia desarrollado por MITRE Corporation que cataloga las tácticas, técnicas y procedimientos (TTPs) utilizados por adversarios en ciberataques reales. Se utiliza en este proyecto para clasificar la actividad observada en el honeypot, proporcionando un lenguaje común que permite comparar hallazgos con la comunidad de ciberseguridad y formular recomendaciones defensivas precisas.

Herramientas de Red y Seguridad

- **iptables y netfilter-persistent:** iptables es la herramienta que gestiona las reglas de netfilter, el sistema de filtrado de paquetes del kernel Linux. En este proyecto cumple dos funciones: en la tabla filter mantiene un «rechazar todo» en la cadena INPUT con excepciones solo para los puertos necesarios, y en la tabla nat una regla PREROUTING redirige el tráfico destinado al puerto 22 hacia el 2222, donde escucha Cowrie, de forma transparente para el atacante. El paquete netfilter-persistent guarda las reglas en /etc/iptables/rules.v4 para que se restauren en cada arranque.
- **OpenSSH:** servidor SSH real para la administración segura del servidor, configurado en el puerto no estándar 2022. Se deshabilita la autenticación por contraseña y el login root, aceptando únicamente claves ed25519. Esta configuración sigue las buenas

prácticas de hardening de SSH recogidas en los benchmarks de CIS (Center for Internet Security).

Decisión de Diseño: iptables en Lugar de UFW

La planificación inicial del proyecto contemplaba UFW (Uncomplicated Firewall) como firewall del host, por ser la herramienta habitual en Ubuntu y por su sintaxis sencilla. UFW no es un firewall distinto, sino una capa de gestión que genera reglas de iptables. Durante la preparación del despliegue, entonces previsto en Oracle Cloud, se analizó la imagen de Ubuntu de esa plataforma y se decidió prescindir de UFW por tres motivos.

En primer lugar, la imagen de Oracle ya incluye un conjunto de reglas de iptables persistentes en `/etc/iptables/rules.v4`, que terminan con un rechazo general del tráfico entrante e incluyen reglas para los servicios internos de la plataforma en el rango 169.254.0.0/16. Activar UFW sobre esa base supondría tener dos gestores escribiendo en las mismas tablas, una fuente habitual de errores difíciles de diagnosticar, como abrir un puerto en UFW y que el tráfico siga bloqueado por una regla previa de Oracle. En el despliegue final en Hetzner la imagen no trae reglas previas, pero sí UFW instalado e inactivo; al instalar iptables-persistent, el gestor de paquetes desinstaló UFW automáticamente por ser incompatibles, lo que confirma que ambas herramientas no deben convivir.

En segundo lugar, la redirección del puerto 22 al 2222 requiere una regla en la tabla nat que UFW no permite crear con sus comandos; obliga a editar a mano su fichero `/etc/ufw/before.rules`. Con iptables es una única regla. Por último, mantener todas las reglas, de filtrado y de NAT, en una sola herramienta simplifica la auditoría: el comando `iptables -S` muestra la configuración completa y activa del sistema.

Un detalle de esta configuración es que el firewall del host debe aceptar el puerto 2222 aunque no se exponga a Internet. La redirección se aplica en la cadena PREROUTING, antes del filtrado, de modo que el paquete llega a la cadena INPUT con el puerto de destino ya cambiado a 2222. El puerto sigue siendo inaccesible desde el exterior porque el Cloud Firewall de Hetzner solo admite el 22 y el 2022.

Herramientas de Desarrollo y Documentación

- **Git / GitHub:** sistema de control de versiones distribuido utilizado para gestionar el código fuente, los scripts y la documentación del

proyecto. El repositorio público (github.com/diegofonterosa/PFC-Cowrie) incluye historial de commits descriptivos, fichero README.md y estructura organizada de directorios.

- **VirtualBox 7.x:** hipervisor de tipo 2 (hosted) de Oracle, utilizado para ejecutar las máquinas virtuales del laboratorio local. Proporciona el aislamiento necesario para analizar datos potencialmente maliciosos sin riesgo para el equipo anfitrión.
- **Kali Linux:** distribución Linux basada en Debian orientada a pruebas de seguridad y pentesting. Se utiliza en este proyecto para realizar ataques controlados contra el honeypot (herramientas como hydra, nmap, ssh) y verificar que Cowrie responde correctamente y genera los logs esperados.
- **matplotlib:** biblioteca Python para la generación de gráficas estáticas, animadas e interactivas. En este proyecto se utiliza para producir gráficas de barras horizontales con los rankings de IPs, contraseñas, usuarios y comandos, exportadas como ficheros PNG de alta resolución.
- **ip-api.com:** API REST gratuita de geolocalización de IPs que devuelve país, ciudad, ISP y coordenadas geográficas para una dirección IP dada. Se consulta en el script de análisis para enriquecer los datos de las IPs atacantes con información geográfica. El tier gratuito permite 45 consultas por minuto.

Metodología

Metodología Utilizada

El proyecto sigue una metodología iterativa e incremental, dividida en ocho fases secuenciales con hitos de control intermedios. Esta aproximación permite avanzar de forma progresiva, validando cada componente antes de construir sobre él. La naturaleza del proyecto —un sistema que debe estar expuesto a Internet para recopilar datos reales— impone una secuencia lógica estricta: primero el despliegue de la infraestructura, después la configuración del honeypot, a continuación la captura de datos durante un período suficiente y finalmente el análisis y la generación del informe.

Esta secuencialidad es inherente al dominio: no se pueden analizar ataques que aún no se han capturado, y no se pueden capturar ataques en un honeypot que aún no se ha desplegado. Sin embargo, dentro de cada fase se permite la iteración: por ejemplo, la configuración de Cowrie puede ajustarse tras observar los primeros ataques, y el script de análisis puede refinarse a medida que se identifican nuevos patrones en los datos.

Cada fase incluye un conjunto de tareas con estimación de horas y un criterio de prueba asociado (estructura RFTP), lo que permite contrastar la planificación inicial con el tiempo real dedicado al finalizar el proyecto. Los hitos de control permiten verificar el avance: el hito del 50% (9 de octubre) coincide con la finalización del despliegue y hardening, mientras que la entrega final se sitúa el 14 de diciembre.

Se ha optado por no utilizar un framework de gestión ágil formal (Scrum, Kanban) al tratarse de un proyecto individual donde la planificación secuencial resulta más natural y eficiente. El seguimiento se realiza mediante el propio repositorio Git, que documenta el progreso a través de los commits, y la tabla RFTP que actúa como checklist de verificación.

Planificación Temporal

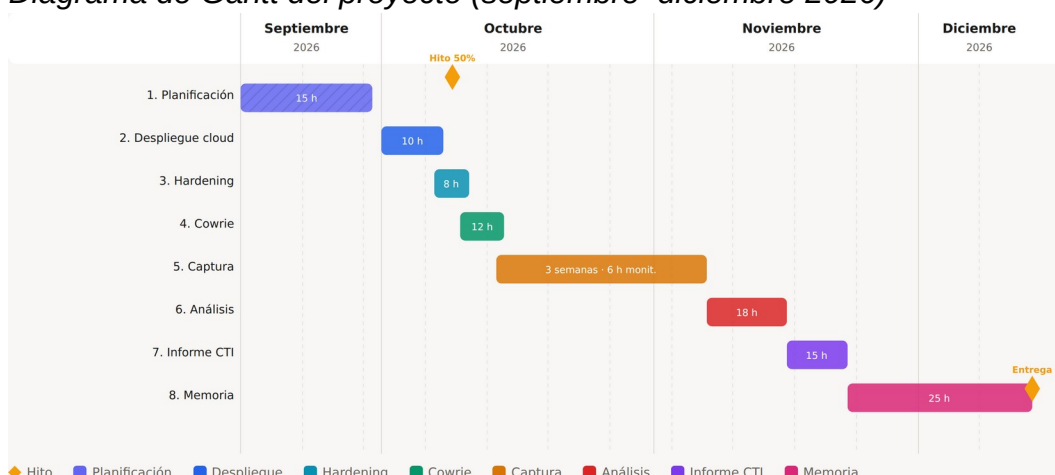
El proyecto se desarrolla entre septiembre y diciembre de 2026, con un total estimado de 109 horas distribuidas en las siguientes fases:

Fase	Descripción	Periodo	Horas	Estado
1	Planificación y documentación inicial	15 sept – 30 sept	15	Completada
2	Despliegue del servidor cloud	1 oct – 7 oct	10	Completada

3	Hardening del servidor	7 oct – 10 oct	8	Completada
4	Instalación y configuración de Cowrie	10 oct – 14 oct	12	Completada
5	Captura de datos (exposición a Internet)	14 oct – 7 nov	6	En curso
6	Análisis con Python	7 nov – 16 nov	18	Script listo
7	Informe de inteligencia (CTI)	16 nov – 23 nov	15	Pendiente
8	Memoria y entrega final	23 nov – 14 dic	25	Pendiente
	Total		109	

Figura 2

Diagrama de Gantt del proyecto (septiembre–diciembre 2026)



Nota. Las barras representan el periodo de cada fase y su carga estimada en horas. Los rombos marcan el hito de control del 50 % (9 de octubre) y la entrega final (14 de diciembre).

Presupuesto

El proyecto utiliza únicamente software libre o gratuito; el único coste es el servidor cloud:

Concepto	Coste	Notas
Servidor Hetzner Cloud CX23 + IPv4	7,25 €/mes	IVA incluido
Software Cowrie	0 €	Open source
Python 3 + bibliotecas	0 €	Open source
Wazuh SIEM	0 €	Open source
VirtualBox	0 €	Open source
Kali Linux	0 €	Open source
API ip-api.com (geolocalización)	0 €	Free tier
Total estimado	≈ 15 €	Unos 2 meses de servidor

Todo el software empleado es libre o gratuito. El único coste económico es el servidor cloud: 6,64 € al mes por el servidor CX23 y 0,61 € por la dirección IPv4, IVA incluido, facturados por horas. Está previsto mantenerlo unos dos meses (despliegue, captura y análisis) y eliminarlo después, junto con su dirección IP, para que deje de facturarse. El anteproyecto contemplaba un coste nulo al usar el tier gratuito de Oracle Cloud; el cambio de proveedor se explica en el capítulo de Tecnología. A ello se suma el tiempo del alumno, estimado en 109 horas.

README y Git

El código fuente, los scripts de análisis y la documentación del proyecto se gestionan en un repositorio público de GitHub:
<https://github.com/diegofonterosa/PFC-Cowrie>

El repositorio incluye un fichero README.md con instrucciones de uso, la estructura de directorios y los comandos necesarios para ejecutar los scripts. Se utiliza Git como sistema de control de versiones, con commits descriptivos que documentan la evolución del proyecto.

La estructura del repositorio se organiza en directorios temáticos:

- **docs/**: anteproyecto y memoria del proyecto en formato editable.
- **scripts/**: scripts Python de análisis (cowrie_analyzer.py) y generación de datos de prueba (generate_sample_logs.py).
- **config/**: ficheros de configuración (Cowrie, SSH, systemd) y scripts de despliegue: bastionado, instalación de Cowrie y redirección con iptables.
- **logs/sample/**: logs de ejemplo para pruebas sin necesidad de datos reales.
- **capturas/**: capturas de pantalla organizadas por fase del proyecto (fase1-despliegue hasta fase6-informe).
- **graficas/**: gráficas PNG generadas por el script de análisis.

Catálogo de Técnicas MITRE ATT&CK

La clasificación de la actividad se automatiza con el módulo mitre_mapping.py, descrito en el capítulo de Implementación. La tabla siguiente recoge su catálogo de reglas: las técnicas que el clasificador es capaz de reconocer y los indicadores que busca en los logs de Cowrie. Los identificadores se han comprobado contra la web oficial de MITRE ATT&CK.

Táctica	Técnica	ID	Indicadores en los logs
Initial Access	Valid Accounts: Local Accounts	T1078.003	Evento cowrie.login.success
Execution	Command and Scripting Interpreter: Unix Shell	T1059.004	Cualquier evento cowrie.command.input
Persistence	Account Manipulation: SSH Authorized Keys	T1098.004	Escritura en authorized_keys
Persistence	Account Manipulation	T1098	chpasswd, passwd, usermod
Persistence	Create Account: Local Account	T1136.001	useradd, adduser
Persistence	Scheduled Task/Job: Cron	T1053.003	crontab -e, crontab -, /etc/cron
Defense Evasion	File and Directory Permissions Modification: Linux	T1222.002	chmod, chattr, chown
Defense Evasion	Indicator Removal: Clear Command History	T1070.003	history -c, unset HISTFILE
Defense Evasion	Indicator Removal: Clear Linux or Mac System Logs	T1070.002	rm de /var/log, wtmp, btmp
Defense Evasion	Deobfuscate/Decode Files or Information	T1140	base64 -d, xxd -r
Defense Evasion	Impair Defenses: Disable or Modify System Firewall	T1562.004	iptables -F, ufw disable
Credential Access	Brute Force: Password Guessing	T1110.001	Evento cowrie.login.failed
Credential Access	OS Credential Dumping: /etc/passwd and /etc/shadow	T1003.008	Lectura de /etc/shadow
Discovery	System Information Discovery	T1082	uname, /proc/cpuinfo, free, nproc, lscpu
Discovery	Account Discovery: Local Account	T1087.001	Lectura de /etc/passwd y /etc/group
Discovery	System Owner/User Discovery	T1033	whoami, id, w, who, last
Discovery	Process Discovery	T1057	ps, top, pgrep, pidof
Discovery	System Network Configuration Discovery	T1016	ifconfig, ip a, route, resolv.conf
Discovery	System Network Connections Discovery	T1049	netstat, ss, lsof -i
Discovery	File and Directory Discovery	T1083	ls, find /
Command and Control	Ingress Tool Transfer	T1105	wget, curl, tftp, ftpget, /dev/tcp
Impact	Resource Hijacking: Compute Hijacking	T1496.001	xmrig, minerd, stratum+tcp, miner

Tabla 6. Catálogo de técnicas MITRE ATT&CK del clasificador.

Una versión anterior de esta tabla, elaborada a mano a partir de la literatura, incluía dos asignaciones que se corrigieron al construir el clasificador. Los intentos de autenticación se habían situado en la táctica Initial Access, cuando la fuerza bruta (T1110) pertenece a Credential Access; y las propias sesiones SSH se habían clasificado como movimiento lateral (T1021.004), cuando el honeypot es el punto de entrada del atacante y no una máquina desde la que se desplace hacia otras. Asimismo, el acceso con las credenciales de root se reasignó de Default Accounts (T1078.001) a Local Accounts (T1078.003).

La tabla con los datos reales (sesiones, IPs y ejemplos de cada técnica) la genera el analizador al procesar los logs de la captura, y se incorporará al informe de inteligencia junto con las recomendaciones defensivas asociadas a cada técnica.

Implementación

Este capítulo describe cómo se ha construido el sistema: los scripts que llevan un servidor Ubuntu recién creado hasta un honeypot operativo, la configuración de Cowrie, los servicios de systemd, el script de análisis y la herramienta de aprovisionamiento desarrollada para intentar obtener la instancia en Oracle Cloud, la plataforma prevista inicialmente. Todo el código se encuentra en el repositorio del proyecto y los ficheros de configuración completos se reproducen en los anexos.

Estrategia de Despliegue

El despliegue se ha dividido en tres scripts de Bash que se ejecutan en orden. La división no es arbitraria: cada script termina en un punto en el que el administrador debe comprobar algo antes de continuar, y un error en ese punto podría dejarle sin acceso al servidor. El riesgo principal del proyecto es precisamente ese: mover el SSH de administración y redirigir el puerto 22 son operaciones que, ejecutadas en mal orden, cortan el único canal de acceso a una máquina remota.

Paso	Script	Función	Comprobación previa
0	Consola del proveedor	Cloud Firewall con entrada TCP 22, TCP 2022 e ICMP	—
1	01_hardening.sh	Usuario administrador, actualizaciones, swap, firewall iptables y SSH real en el puerto 2022	Hay una clave autorizada; el 2022 está abierto en el firewall del proveedor
2	02_instalar_cowrie.sh	Usuario cowrie, Cowrie 3.1.0, configuración, servicios systemd	userdb.txt es ASCII puro
3	03_redireccion_22.sh	Redirección NAT 22 → 2222	sshd escucha en 2022 y Cowrie está activo

Tabla 7. Scripts de despliegue y comprobaciones de seguridad.

Todos los scripts son idempotentes: antes de crear una regla de firewall comprueban si ya existe (iptables -C), y antes de inicializar Cowrie comprueban si ya hay configuración. Así pueden volver a ejecutarse tras un fallo parcial sin duplicar reglas ni sobrescribir cambios. Además, se ejecutan con set -euo pipefail, de modo que cualquier orden que falle detiene el script en lugar de continuar con un estado inconsistente.

Bastionado del Servidor

El script `01_hardening.sh` aplica el bastionado básico del sistema. Comienza actualizando los paquetes y activando `unattended-upgrades` para que los parches de seguridad se instalen automáticamente durante las semanas de exposición. Si la máquina tiene menos de 2 GB de RAM, crea además un fichero de swap de 2 GB, previsto para el caso de que hubiera que recurrir a una instancia pequeña.

Usuario Administrador

Como el script prohíbe el acceso SSH como root, antes debe existir un usuario administrador con una clave autorizada; de lo contrario, el administrador se quedaría fuera del servidor. Oracle y AWS entregan sus máquinas con un usuario `ubuntu`, pero Hetzner solo permite entrar como root. El script detecta el caso: si se ejecuta con `sudo` usa el usuario que lo invoca; si existe `ubuntu` con clave, usa ese; y si solo hay root, crea un usuario nuevo (diego en este proyecto), le copia la clave autorizada de root con los permisos correctos (700 para el directorio `.ssh` y 600 para el fichero), lo añade al grupo `sudo` y le pide una contraseña que solo se usará para `sudo`, ya que el acceso SSH sigue siendo exclusivamente por clave. Para poder relanzar el script tras un fallo, la contraseña solo se solicita si el usuario aún no tiene una.

Configuración de OpenSSH

La configuración del SSH real no se aplica editando `/etc/ssh/sshd_config`, sino mediante un fichero independiente en el directorio `/etc/ssh/sshd_config.d/`. Esta decisión responde a dos particularidades de OpenSSH que se detectaron durante el diseño:

- **Gana el primer valor, no el último:** para la mayoría de opciones, `sshd` usa la primera aparición que encuentra. Las imágenes cloud de Ubuntu incluyen ficheros como `50-cloud-init.conf` en ese directorio, así que el fichero del proyecto se nombra `00-pfc-hardening.conf` para leerse antes que todos ellos.
- **La opción Port se acumula:** a diferencia del resto, cada línea `Port` añade un puerto de escucha. Si `sshd_config` conserva un «`Port 22`», el SSH real seguiría escuchando en el 22 y la redirección posterior entraría en conflicto. El script comenta cualquier línea `Port` del fichero principal.

El fichero establece el puerto 2022, la autenticación exclusiva por clave pública (AuthenticationMethods publickey), la prohibición del acceso como root, la restricción a un único usuario administrador (AllowUsers) y límites de tres intentos y 30 segundos para autenticarse. Antes de reiniciar el servicio se valida la sintaxis con `sshd -t`; si falla, el script elimina el fichero y se detiene sin tocar el servicio en ejecución. Esta validación necesita que exista el directorio `/run/sshd`, que en Ubuntu 24.04 solo se crea mientras el servicio está en marcha; por eso el script lo crea antes de validar (véase la incidencia correspondiente en el capítulo de Pruebas).

Ubuntu 24.04 introduce otra particularidad: el SSH se arranca por activación de socket (`ssh.socket`) y es `systemd`, no `sshd`, quien abre el puerto de escucha. Por eso, tras cambiar el puerto no basta con reiniciar `ssh.service`; hay que recargar la configuración de `systemd` y reiniciar `ssh.socket`. El script hace ambas cosas y después comprueba con `ss` que hay un proceso escuchando en el 2022. Si no lo hay, se detiene y avisa de no cerrar la sesión actual.

Firewall con iptables

El script detecta si la cadena INPUT ya termina en una regla de rechazo, como ocurre en las imágenes de Oracle. Si no es así, como en Hetzner, construye la base de una política restrictiva: aceptar el tráfico de conexiones establecidas, la interfaz de loopback, ICMP y el puerto 22, y rechazar todo lo demás. Sobre esa base inserta la regla que abre el 2022 y guarda el conjunto con `netfilter-persistent`. El puerto 22 se mantiene abierto en este paso porque la sesión del administrador sigue entrando por él hasta que se complete el despliegue.

```
ensure() { # inserta la regla al principio solo si no existe ya
    local chain=$1; shift
    iptables -C "$chain" "$@" 2>/dev/null || iptables -I "$chain" 1 "$@"
}
ensure INPUT -p tcp --dport "$SSH_PORT" -m conntrack --ctstate NEW -j ACCEPT
netfilter-persistent save
```

Instalación y Configuración de Cowrie

Modelo de Instalación de Cowrie 3.x

Durante la preparación del proyecto se comprobó que Cowrie cambió su modelo de instalación a partir de la versión 3.0. Las versiones anteriores se instalaban clonando el repositorio y ejecutando `bin/cowrie`

desde el propio código fuente, que es lo que describe la mayoría de tutoriales disponibles. Desde la versión 3 se instala como un paquete de Python (`pip install cowrie`) y se inicializa un directorio de estado con `cowrie init`, que crea la configuración en `etc/` y las carpetas de logs, descargas y claves en `var/`. El proyecto fija la versión 3.1.0 para que el despliegue sea reproducible.

Cowrie se ejecuta con un usuario dedicado sin privilegios (`cowrie`) y dentro de un entorno virtual de Python, de forma que sus dependencias no interfieren con las del sistema. El propio Cowrie se niega a arrancar como `root`, una medida de seguridad coherente con su función: es un programa expuesto deliberadamente a atacantes.

Configuración por Capas

Cowrie 3 carga primero sus valores por defecto y después el fichero `etc/cowrie.cfg`, que solo necesita contener las claves que se quieren cambiar. El fichero del proyecto (Anexo A) modifica los siguientes aspectos:

Opción	Valor por defecto	Valor del proyecto	Motivo
hostname	svr04	web-prod-02	svr04 es una firma conocida de Cowrie
version (SSH)	OpenSSH_9.2p1 Debian-2+deb12u3	OpenSSH_9.2p1 Debian-2+deb12u5	Evitar la firma por defecto
kernel_version	6.1.0-21-amd64	6.1.0-31-amd64	Evitar la firma por defecto
arch	linux-x64-lsb	linux-x64-lsb	x86_64 es la arquitectura que más malware recibe
logtype	rotating	plain	Un único fichero JSON para el análisis
telnet	desactivado	desactivado	Fuera del alcance

Tabla 8. Opciones de Cowrie modificadas.

Las firmas por defecto son relevantes porque algunos bots comprueban el nombre de la máquina, la versión del kernel o el banner SSH para detectar honeypots y abandonar la sesión. La identidad elegida es la de un Debian 12 actualizado, coherente con el sistema de ficheros simulado que incluye Cowrie. La arquitectura simulada es `x86_64`, la misma que la del servidor real de Hetzner (CX23, `x86_64`, según confirma el mensaje de bienvenida de Ubuntu al conectarse). La coincidencia no es imprescindible, porque el atacante interactúa con el sistema falso y no con el real, pero evita incoherencias si un bot compara la arquitectura

declarada con otras señales. Además, x86_64 es la arquitectura más común en los scripts de los atacantes, lo que favorece la captura de malware.

Estrategia de Credenciales

El fichero etc/userdb.txt (Anexo B) decide qué combinaciones de usuario y contraseña acepta el honeypot. Se evalúa línea a línea y gana la primera coincidencia. La estrategia elegida rechaza las contraseñas más probadas (root, 123456, password, admin, ubuntu) y acepta cualquier otra para los usuarios root, admin y ubuntu. El objetivo es doble: que los bots sigan probando y registren más credenciales, y que finalmente entren para poder capturar los comandos que ejecutan tras el acceso, que es la información más valiosa para el análisis de TTPs.

Servicio de systemd Confinado

Cowrie se gestiona como servicio de systemd (Anexo D) en primer plano, gracias a la variable COWRIE_STDOUT=yes, que hace que su log vaya al journal del sistema. El servicio se reinicia automáticamente si falla y aplica opciones de confinamiento: sistema de ficheros en solo lectura salvo el directorio var/ de Cowrie, sin capacidades del kernel, sin posibilidad de ganar privilegios y con un límite de 512 MB de memoria. Se comprobó que Cowrie solo escribe en var/ (logs, claves SSH del host, sesiones TTY y descargas), por lo que estas restricciones no afectan a su funcionamiento. El objetivo es que, aun en el caso improbable de que un atacante lograra escapar del entorno simulado, no pudiera modificar el sistema.

Reserva de Memoria

En instancias A1 de Oracle, el script de instalación activa además el servicio oci-keepalive (Anexo E), descrito en el apartado de Tecnología; en el despliegue final en Hetzner no se activa. Su implementación tiene un detalle técnico relevante: reservar memoria con bytearray() en Python no basta, porque Linux no asigna páginas físicas a memoria que solo se ha reservado y nunca se ha escrito. El programa escribe un byte en cada página de 4 KB para que toda la reserva ocupe RAM real, y el servicio prohíbe el uso de swap (MemorySwapMax=0), ya que la memoria movida a swap no contaría como utilizada.

Exposición del Honeypot

El script 03_redireccion_22.sh es el último paso y el que convierte el servidor en un honeypot público. Antes de actuar comprueba que el SSH real escucha en el 2022 y que Cowrie está activo; si alguna condición falla, se detiene. Después añade dos reglas: la redirección NAT del 22 al 2222 en la cadena PREROUTING y la aceptación del 2222 en INPUT, necesaria porque la redirección ocurre antes del filtrado. Las conexiones ya establecidas por el 22 no se ven afectadas, ya que la NAT solo se aplica a conexiones nuevas, de modo que la sesión del administrador no se corta al ejecutarlo.

```
ensure filter INPUT -p tcp --dport 2222 -m conntrack --ctstate NEW -j ACCEPT
ensure nat PREROUTING -p tcp --dport 22 -j REDIRECT --to-port 2222
netfilter-persistent save
```

Script de Análisis

El script cowrie_analyzer.py procesa uno o varios ficheros JSON de Cowrie y está organizado en funciones independientes por tipo de análisis, lo que permite añadir métricas nuevas sin modificar las existentes:

Función	Eventos de Cowrie	Resultado
analyze_connections	cowrie.session.connect	IPs únicas y número de sesiones
analyze_logins	cowrie.login.failed / success	Rankings de usuarios, contraseñas y combinaciones
analyze_commands	cowrie.command.input	Comandos más ejecutados
analyze_downloads	cowrie.session.file_download	URLs descargadas y hash SHA-256
analyze_command_urls	cowrie.command.input	URLs en comandos aunque la descarga fallase
analyze_timeline	cowrie.session.connect	Actividad diaria
geolocate_ips	—	País, ciudad e ISP de las IPs (ip-api.com)
mappear (mitre_mapping.py)	login.failed / success, command.input	Técnicas MITRE ATT&CK con evidencias

Tabla 9. Funciones del analizador.

La función analyze_command_urls se añadió tras las pruebas con Cowrie real. Cowrie solo registra un evento de descarga cuando el fichero se descarga con éxito, pero en la práctica muchos servidores de malware de las botnets ya han sido retirados cuando el bot ejecuta el comando wget o curl. Esas URLs siguen siendo indicadores de compromiso (IOC)

útiles para el informe de inteligencia, así que la función las extrae directamente del texto de los comandos mediante una expresión regular.

Clasificación Automática en MITRE ATT&CK

La clasificación de la actividad observada se ha automatizado en un módulo independiente, `mitre_mapping.py`, que el analizador utiliza en cada ejecución. Hacerlo a mano sobre miles de eventos sería lento y propenso a errores; automatizarlo permite además repetir el análisis cada vez que se descargan logs nuevos durante el periodo de captura.

El módulo define un catálogo de reglas (Tabla 6), cada una formada por el identificador de la técnica, su nombre oficial, la táctica a la que pertenece, una expresión regular que reconoce los indicadores en el texto del comando y una recomendación defensiva, con la referencia a la mitigación de ATT&CK (M1022, M1027, M1029...) cuando el marco la define. Las decisiones de diseño más relevantes son:

- **Un comando puede corresponder a varias técnicas.** Los bots suelen encadenar órdenes en una sola línea (`cd /tmp; wget ...; chmod +x ...; ./bot`). Cada regla se evalúa por separado sobre el comando completo, de modo que esa línea suma a la vez a T1105, T1222.002 y T1059.004.
- **La autenticación se clasifica por tipo de evento, no por texto.** Los eventos `cowrie.login.failed` se asignan a T1110.001 (fuerza bruta) y los `cowrie.login.success` a T1078.003 (uso de cuentas válidas), con la credencial empleada como evidencia.
- **Se cuentan sesiones e IPs, no solo eventos.** Un mismo bot puede repetir un comando muchas veces en una sesión. El número de sesiones distintas y de IPs distintas que usan cada técnica refleja mejor su prevalencia real entre los atacantes.
- **Lo que no se clasifica se muestra.** Los comandos que no encajan en ninguna regla aparecen en una sección propia del informe para su revisión manual. Así, una técnica que el catálogo no contemple no se pierde en silencio, y la revisión permite ampliar las reglas.

El resultado se entrega en tres formatos: una sección del informe de texto agrupada por tácticas en el orden de la matriz de ATT&CK, una gráfica de sesiones por técnica y un fichero CSV (`mitre_mapping.csv`) con táctica, identificador, nombre, sesiones, eventos, IPs, ejemplo y

recomendación, que se abre directamente en una hoja de cálculo y sirve de base para las tablas del informe de inteligencia.

Herramienta de Aprovisionamiento en OCI

El script `oci_a1_retry.py` se desarrolló para resolver la falta de capacidad de instancias A1 en la región. Utiliza el SDK oficial de Oracle para Python y se autentica con una clave de API asociada al usuario, cuyo fichero privado permanece en el equipo del administrador y está excluido del repositorio mediante `.gitignore`. Aunque finalmente Oracle no tuvo capacidad disponible y el proyecto se desplegó en Hetzner Cloud, la herramienta se documenta porque formó parte del trabajo realizado y es reutilizable por cualquier usuario del tier gratuito de Oracle.

Su funcionamiento se divide en dos fases. En la primera descubre por sí mismo los datos necesarios: el dominio de disponibilidad, la imagen Ubuntu 24.04 más reciente compatible con A1 (prefiriendo la imagen completa a la mínima) y la subnet pública. Comprueba también si ya existe una instancia con el nombre `honeypot-cowrie` para no crear duplicados. En la segunda fase solicita la instancia en bucle y trata cada respuesta de la API de forma diferente:

Respuesta de OCI	Significado	Acción del script
500 «Out of host capacity»	No hay máquinas libres	Reintentar tras el intervalo (60 s)
429 TooManyRequests	Demasiadas peticiones	Duplicar la espera, hasta 10 minutos
Otros 5xx	Error temporal de Oracle	Reintentar
4xx (límites, permisos)	Error de configuración	Detenerse y avisar
Éxito	Instancia creada	Esperar a RUNNING, mostrar IP y avisar

Tabla 10. Tratamiento de las respuestas de la API de OCI.

A cada espera se suma un retardo aleatorio de hasta 10 segundos para no enviar peticiones a intervalos exactos. Opcionalmente, el script envía notificaciones push al móvil mediante el servicio ntfy al arrancar, al conseguir la instancia y si se detiene por un error, de modo que el administrador no necesita vigilar la consola. Un fallo al enviar una notificación nunca interrumpe el script.

Normalización de Saltos de Línea

El desarrollo se realiza desde Windows, donde Git convierte por defecto los saltos de línea de los ficheros de texto al formato de Windows (CRLF). Un script de Bash con ese formato falla en Linux con errores poco descriptivos, como «\$'\r': command not found». Para evitarlo, el repositorio incluye un fichero `.gitattributes` que fuerza el formato de Linux (LF) en los scripts y ficheros de configuración, y el procedimiento de despliegue clona el repositorio directamente en el servidor en lugar de copiar los ficheros desde Windows.

Pruebas

Las pruebas se han realizado en dos etapas. La primera valida en un entorno Linux local los componentes que no dependen del servidor cloud: la instalación y configuración de Cowrie, su comportamiento ante un ataque, el analizador y las herramientas auxiliares. La segunda corresponde a las pruebas de la tabla RFTP sobre el servidor desplegado en Hetzner Cloud. Este capítulo documenta ambas etapas y las incidencias detectadas en cada una.

Entorno de Pruebas Local

Las pruebas locales se ejecutaron en una máquina Linux con Python 3.11, en la que se instaló Cowrie 3.1.0 siguiendo exactamente el mismo procedimiento que aplica el script 02_instalar_cowrie.sh: usuario dedicado sin privilegios, entorno virtual, instalación desde PyPI, inicialización con `cowrie init` y copia de los ficheros `cowrie.cfg` y `userdb.txt` del proyecto. Cowrie se arrancó en primer plano con `COWRIE_STDOUT=yes`, el mismo modo que utiliza el servicio de `systemd`.

Prueba de Arranque

Cowrie arrancó sin errores, cargó el módulo de salida JSON y quedó escuchando en el puerto 2222. Al conectarse al puerto, el banner recibido fue `SSH-2.0-OpenSSH_9.2p1 Debian-2+deb12u5`, es decir, la versión configurada en `cowrie.cfg` y no la que Cowrie trae por defecto. Esto confirma que el fichero de configuración se carga correctamente por encima de los valores predeterminados.

Incidencia: Rechazo Silencioso de Todos los Logins

En la primera prueba de autenticación, Cowrie rechazó todas las credenciales, incluidas las que `userdb.txt` debía aceptar. El servicio no mostró ningún error al arrancar y seguía respondiendo con normalidad a las conexiones, por lo que desde fuera el honeypot parecía funcionar.

La revisión del log de Cowrie mostró una excepción `UnicodeDecodeError` en el módulo de autenticación: Cowrie 3.1.0 lee el fichero `userdb.txt` con la codificación ASCII, y los comentarios del fichero contenían tildes y otros caracteres no ASCII. Al fallar la lectura, el módulo no puede cargar ninguna regla y deniega cualquier intento. El error solo

aparece en el log en el momento de cada intento de autenticación, no al arrancar.

La gravedad de este fallo reside en que es silencioso. Si se hubiese desplegado así, el honeypot habría registrado durante semanas los intentos de fuerza bruta (que se generan antes de la autenticación) pero ningún acceso ni ningún comando, que son precisamente los datos necesarios para el análisis de TTPs y el mapeo MITRE ATT&CK. Probablemente no se habría detectado hasta la fase de análisis, sin margen para repetir la captura.

La corrección fue reescribir los comentarios del fichero sin caracteres no ASCII. Como medida preventiva, el script de instalación verifica ahora la codificación del fichero antes de instalarlo y se detiene si encuentra algún carácter no ASCII. El propio fichero incluye un aviso en su cabecera.

Ataque Simulado

Con la configuración corregida se ejecutó un ataque simulado mediante un script de Python con la biblioteca Paramiko, reproduciendo el patrón típico de un bot: varios intentos de fuerza bruta con credenciales comunes y, tras obtener acceso, comandos de reconocimiento, un intento de descarga de malware y el borrado del historial.

Usuario	Contraseña	Resultado esperado	Resultado obtenido
root	root	Rechazo (regla !root)	Rechazado
root	123456	Rechazo (regla !123456)	Rechazado
admin	admin	Rechazo (regla !admin)	Rechazado
oracle	oracle	Rechazo (usuario no aceptado)	Rechazado
root	Passw0rd!	Acceso (regla root:*)	Acceso concedido

Tabla 11. Resultados de la prueba de autenticación.

Una vez dentro, el sistema simulado respondió de forma verosímil a los comandos del atacante. El comando `uname -a` devolvió la identidad configurada («Linux web-prod-02 6.1.0-31-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.128-1 (2025-02-07) x86_64 GNU/Linux»), `cat /etc/passwd` mostró un fichero de usuarios falso, `whoami` respondió `root` y `/proc/cpuinfo` describió un procesador Intel Xeon. El intento de descarga con `wget` falló, como se esperaba, porque el entorno de prueba no tenía salida a esa dirección, y el comando `history -c` se registró aunque el atacante lo usara para borrar su rastro.

Validación del Analizador con Datos Reales

El script de análisis se había desarrollado y probado inicialmente con 500 sesiones generadas por `generate_sample_logs.py`. Esta prueba lo valida contra la salida real de Cowrie 3.1.0, para confirmar que los nombres de los eventos y sus campos coinciden con los que espera el script. El fichero `cowrie.json` generado por el ataque simulado contenía 58 eventos:

Tipo de evento	Número
<code>cowrie.session.connect</code>	11
<code>cowrie.session.closed</code>	11
<code>cowrie.client.version</code>	10
<code>cowrie.client.kex</code>	10
<code>cowrie.command.input</code>	8
<code>cowrie.login.failed</code>	4
<code>cowrie.login.success</code>	1
<code>cowrie.client.size</code>	1
<code>cowrie.session.params</code>	1
<code>cowrie.log.closed</code>	1

Tabla 12. Eventos registrados por Cowrie en el ataque simulado.

El analizador procesó el fichero sin errores y obtuvo 11 sesiones desde 1 IP, 5 intentos de autenticación (1 exitoso, un 20 %), 8 comandos y ninguna descarga completada, cifras que coinciden con la tabla anterior. Generó además las cinco gráficas y el informe de resumen. Esta prueba puso de manifiesto la limitación descrita en el capítulo de Implementación: la URL del `wget` fallido no aparecía en ninguna parte del informe. Tras añadir la función `analyze_command_urls`, el informe incluye una sección de URLs en comandos con esa dirección. Se volvió a ejecutar el análisis sobre los datos simulados para comprobar que la mejora no alteraba el resto de resultados.

Prueba del Clasificador MITRE ATT&CK

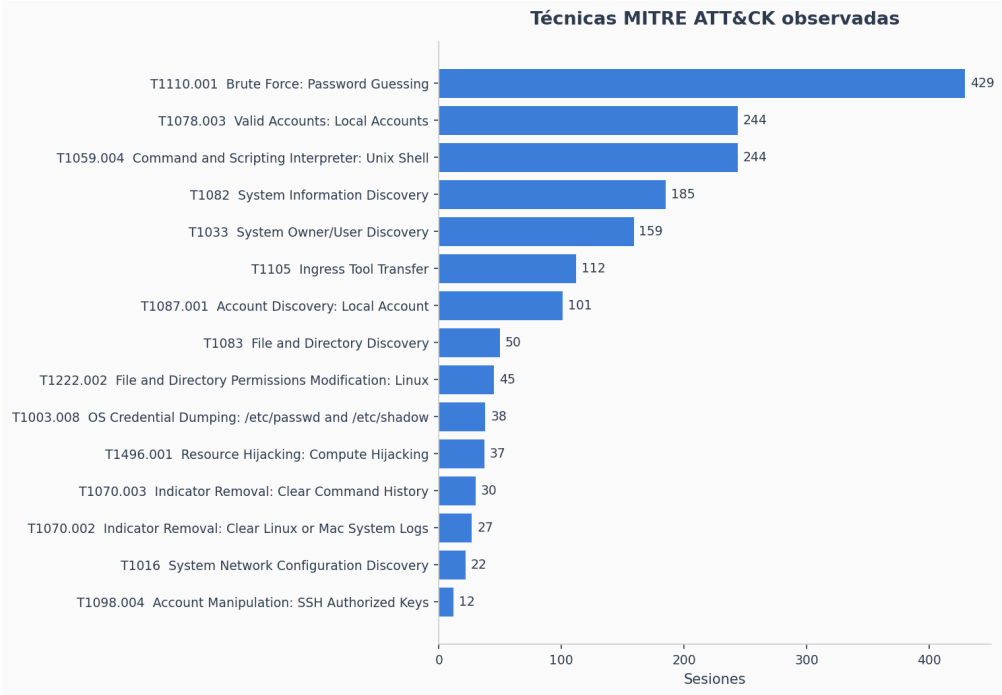
El clasificador se prueba con un conjunto de 33 casos recogido en el fichero `test_mitre_mapping.py`, que puede ejecutarse de nuevo cada vez que se modifiquen las reglas. Los casos incluyen los comandos típicos de los bots y, sobre todo, posibles falsos positivos: las expresiones regulares cortas son propensas a encontrar coincidencias donde no las hay.

Comando	Riesgo	Técnicas obtenidas
wget http://.../bot.x86 -O /tmp/bot	Que la «w» se confunda con el comando w (T1033)	Solo T1105
pidof sshd	Que contenga «id» y cuente como T1033	Solo T1057
crontab -l	Que listar tareas cuente como persistencia	Ninguna propia
(crontab -l; echo '...') crontab -	Que no se detecte la instalación de la tarea	T1053.003
curl http://.../miner.sh -o /tmp/miner.sh	Que solo se detecte la descarga	T1105 y T1496.001
echo hidden / cat /tmp/window.txt	Coincidencias parciales («id», «w»)	Ninguna

Tabla 13. Casos de prueba del clasificador con riesgo de error.

Los 33 casos se clasificaron correctamente. A continuación se ejecutó el analizador completo sobre las 500 sesiones simuladas: identificó 15 técnicas pertenecientes a 8 tácticas, y solo tres comandos quedaron sin técnica propia (cd /tmp, crontab -l y la ejecución directa de /tmp/bot), que efectivamente no corresponden a ninguna técnica distinta de la ejecución en shell. La Figura 4 muestra la gráfica generada. Debe interpretarse solo como validación del funcionamiento, ya que los datos son sintéticos; los resultados reales se presentarán en el informe de inteligencia.

Figura 4
Técnicas MITRE ATT&CK en los datos simulados (sesiones por técnica)



Prueba de la Herramienta de Aprovisionamiento

El script `oci_a1_retry.py` no puede probarse de forma controlada contra Oracle, ya que el éxito depende de que haya capacidad libre. Se probó sustituyendo los clientes del SDK por objetos simulados que devuelven respuestas preparadas, una técnica habitual en pruebas unitarias, y levantando un servidor HTTP local que hace las veces de ntfy y registra las notificaciones recibidas:

Escenario simulado	Comportamiento esperado	Resultado
Dos respuestas «Out of host capacity»	Reintentar cada 60 s	Correcto
Respuesta 429 TooManyRequests	Duplicar la espera a 120 s	Correcto
Éxito en el cuarto intento	Mostrar IP y enviar aviso urgente	Correcto
Imágenes completa y mínima disponibles	Elegir la completa	Correcto
Error 400 LimitExceeded	Detenerse con código 1 y avisar	Correcto
Servidor de notificaciones caído	Registrar el fallo y continuar	Correcto

Tabla 14. Pruebas del script de aprovisionamiento.

El script se ejecutó contra la cuenta real del proyecto los días 3 y 4 de octubre de 2026. Las peticiones confirmaron que la autenticación con

la clave de API, el descubrimiento de recursos y el tratamiento del error de capacidad funcionan contra la API real, aunque en más de 600 intentos Oracle no llegó a tener capacidad disponible, lo que motivó el cambio de proveedor.

Prueba de la Reserva de Memoria

El programa `oci-keepalive.py` se ejecutó reservando un 5 % de la memoria de la máquina de pruebas (7,8 GiB). Informó de una reserva de 402 MiB y el tamaño residente del proceso (VmRSS) fue de 420 MB, lo que confirma que la memoria está ocupada físicamente y no solo reservada. Al recibir la señal SIGTERM, que es la que envía systemd para detener un servicio, el proceso terminó de forma ordenada en un par de segundos, el tiempo necesario para liberar la memoria.

Despliegue en Hetzner Cloud

El 4 de octubre de 2026 se creó el servidor en Hetzner Cloud y se ejecutó el script de bastionado. Ese mismo día se instaló Cowrie y se activó la redirección del puerto 22, con lo que comenzó la captura. El despliegue real sacó a la luz tres incidencias que las pruebas locales no detectaron: dos dependían del sistema del proveedor y la tercera, del entorno en que systemd ejecuta los servicios.

Incidencia: Validación de sshd en Ubuntu 24.04

En la primera ejecución, el script se detuvo al configurar el SSH con el mensaje «Missing privilege separation directory: /run/sshd» y deshizo el cambio. La configuración era correcta: el fallo se debía a que Ubuntu 24.04 arranca sshd por activación de socket, y el directorio /run/sshd solo existe mientras el servicio está en marcha. La validación previa con `sshd -t` lo exige, y al no encontrarlo daba la configuración por errónea.

El comportamiento del script en ese momento fue el diseñado: ante cualquier duda sobre la configuración del SSH, revertir el cambio y no reiniciar el servicio, de modo que la sesión del administrador nunca llegó a correr peligro. La corrección consistió en crear el directorio antes de validar. Al relanzar el script, completó el resto de pasos sin repetir los ya hechos, gracias a su diseño idempotente.

Incidencia: Clave SSH sin Frase Conocida

El servidor se creó inicialmente con una clave SSH existente en el equipo del administrador cuya frase de paso no se recordaba, por lo que no fue posible acceder. Como el servidor estaba recién creado y vacío, la solución más segura fue generar una clave ed25519 nueva y dedicada al proyecto, guardar su frase en el gestor de contraseñas en el momento de crearla y volver a crear el servidor con ella. La lección aplicada es usar una clave distinta por proyecto o servicio, con su frase registrada desde el principio.

Incidencia: Cowrie no Arrancaba como Servicio

El script de instalación terminó con el mensaje «Cowrie no arranca», gracias a la comprobación final que verifica que el servicio está activo y escucha en el puerto 2222. El registro del servicio mostraba un `FileNotFoundError`: el programa `cowrie start` lanza a su vez `twistd` buscándolo en el `PATH`, y `systemd` no activa el entorno virtual de Python donde está instalado. En las pruebas locales no apareció porque Cowrie se arrancaba con el entorno virtual ya activado.

Corregido el `PATH`, apareció un segundo fallo que el primero ocultaba: en primer plano, `twistd` escribe por defecto un fichero `twistd.pid` en el directorio de trabajo, y el confinamiento del servicio (`ProtectHome=read-only`) deja ese directorio en solo lectura. Ambos fallos se reprodujeron en local con Cowrie 3.1.0 antes de corregirlos. La solución fue que el servicio ejecute directamente `twistd` con su ruta completa y la opción `--pidfile=` vacía, ya que `systemd` conoce el PID del proceso y el fichero no aporta nada. Así se mantuvo intacto el confinamiento, en lugar de relajarlo para que el servicio arrancara.

La incidencia confirma el valor de que los scripts comprueben el resultado y no solo ejecuten pasos: sin esa comprobación, la instalación habría parecido correcta y el honeypot habría expuesto un puerto sin servicio.

Resultados del Bastionado

Tras la segunda ejecución del script, el usuario `diego` accede por el puerto 2022 con la clave dedicada. Al instalar `iptables-persistent`, el sistema desinstaló automáticamente `UFW`, que venía instalado pero

inactivo en la imagen de Hetzner, lo que confirma la incompatibilidad entre ambas herramientas comentada en el capítulo de Tecnología.

Resultados de la Instalación de Cowrie y la Exposición

Las pruebas realizadas en el servidor el 4 de octubre, todas con captura de pantalla, fueron las siguientes:

- El acceso SSH como root al puerto 2022 es rechazado (R01F02).
- Tras reiniciar el servidor, las reglas de iptables se mantienen y el SSH real solo escucha en el puerto 2022.
- Prueba local de Cowrie en el puerto 2222: la contraseña root es rechazada, la contraseña prueba da acceso con la identidad web-prod-02, y los intentos y comandos quedan registrados en cowrie.json.
- Prueba externa desde el equipo del administrador: la conexión al puerto 22 aterriza en Cowrie con la identidad web-prod-02, y el registro en directo muestra el login y los comandos con la IP pública de origen (R02F01T03P01).

Las sesiones de prueba quedan en el registro junto con los ataques reales. Para que no alteren las estadísticas, el analizador excluye siempre las conexiones desde localhost y admite la opción `--exclude-ip` para descartar la IP pública del administrador.

Escaneo Externo con nmap

El 7 de octubre se escanearon desde el exterior los puertos 1 a 3000 del servidor con nmap 7.98 (`nmap -Pn -sT -sV`), para comprobar qué ve un atacante (R01F02T01P01):

Puerto	Estado	Servicio identificado por nmap
22/tcp	Abierto	OpenSSH 9.2p1 Debian 2+deb12u5 (Cowrie)
2022/tcp	Abierto	OpenSSH 9.6p1 Ubuntu 3ubuntu13.19 (SSH real)
Resto (2998)	Filtrado, sin respuesta	—

Tabla 15. Resultado del escaneo externo con nmap.

El resultado confirma tres aspectos. Solo los dos puertos previstos son visibles. Cowrie supera la detección de versiones de nmap, que lo identifica como el OpenSSH de un Debian 12, la identidad configurada. Y el motivo «no-response» de los puertos filtrados demuestra que el Cloud Firewall de Hetzner está aplicado: si solo actuase el iptables del servidor,

la regla REJECT devolvería un mensaje ICMP y nmap mostraría otro motivo.

El escaneo revela también una debilidad: el puerto 2022 se identifica como Ubuntu y el 22 como Debian, de modo que un atacante que compare ambos puede deducir que en la misma IP conviven dos sistemas distintos y sospechar del honeypot. La mitigación propuesta es restringir en el Cloud Firewall el puerto 2022 a la IP del administrador.

Pruebas Pendientes en el Servidor

Queda pendiente la siguiente prueba de la tabla RFTP, que se documentará con sus capturas:

- Ataque controlado desde Kali Linux con hydra y verificación en cowrie.json (CU-03).

Resultados

Este capítulo presenta los resultados de los primeros días de captura, del 4 de octubre de 2026 a las 15:49 UTC (primera conexión hostil, 8 minutos después de exponer el puerto 22) al 7 de octubre a las 16:45 UTC. Son resultados parciales: la captura continúa hasta finales de octubre y el informe de inteligencia final se elaborará con el periodo completo. Las cifras se han obtenido con `cowrie_analyzer.py` excluyendo las sesiones de prueba del administrador.

Volumen de la Actividad

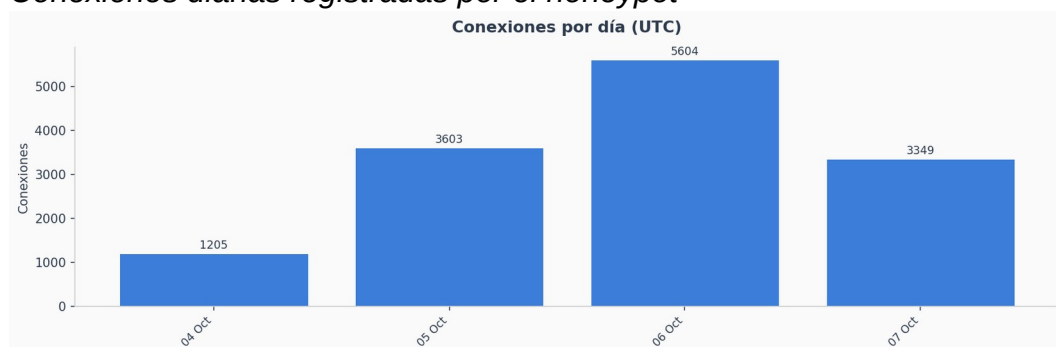
Indicador	Valor
Conexiones (sesiones)	13 761
IPs de origen distintas	499
Intentos de autenticación	12 748 (10 688 fallidos y 2 060 aceptados)
Comandos ejecutados	2 657
Ficheros capturados	654 eventos de 17 ficheros distintos
Técnicas MITRE ATT&CK identificadas	21, en 9 tácticas

Tabla 16. Resumen de la actividad capturada (4–7 de octubre de 2026).

El primer acceso con éxito se produjo a los 26 minutos de la exposición. La media es de unas 3 400 conexiones diarias, repartidas a todas las horas del día. La Figura 5 muestra la actividad por día; el primero y el último son parciales.

Figura 5

Conexiones diarias registradas por el honeypot



Naturaleza Automatizada de los Ataques

Todos los indicadores apuntan a herramientas automatizadas y no a operadores humanos. En las sesiones con comandos, la mediana entre

la autenticación y el primer comando es de 0,7 segundos, y la mitad de todas las sesiones dura menos de 1,4 segundos. El 85,2 % de las conexiones se identifican con el cliente SSH-2.0-Go, una biblioteca de programación y no un cliente interactivo; le siguen libssh (12,8 %) y, de forma testimonial, PuTTY (0,8 %). La Tabla 17 clasifica las sesiones según lo que llegó a hacer el atacante.

Comportamiento	Sesiones	%
Prueba credenciales y no consigue entrar	10 509	76,4
Se conecta sin intentar autenticarse (escaneo)	1 192	8,7
Entra y ejecuta comandos o sube ficheros	1 559	11,3
Entra y se desconecta sin hacer nada	501	3,6

Tabla 17. Clasificación de las sesiones por comportamiento.

Las 501 sesiones que se desconectan tras entrar corresponden a la verificación de credenciales: el bot comprueba que la contraseña funciona y la registra para un uso posterior, posiblemente por otro actor.

Origen Geográfico

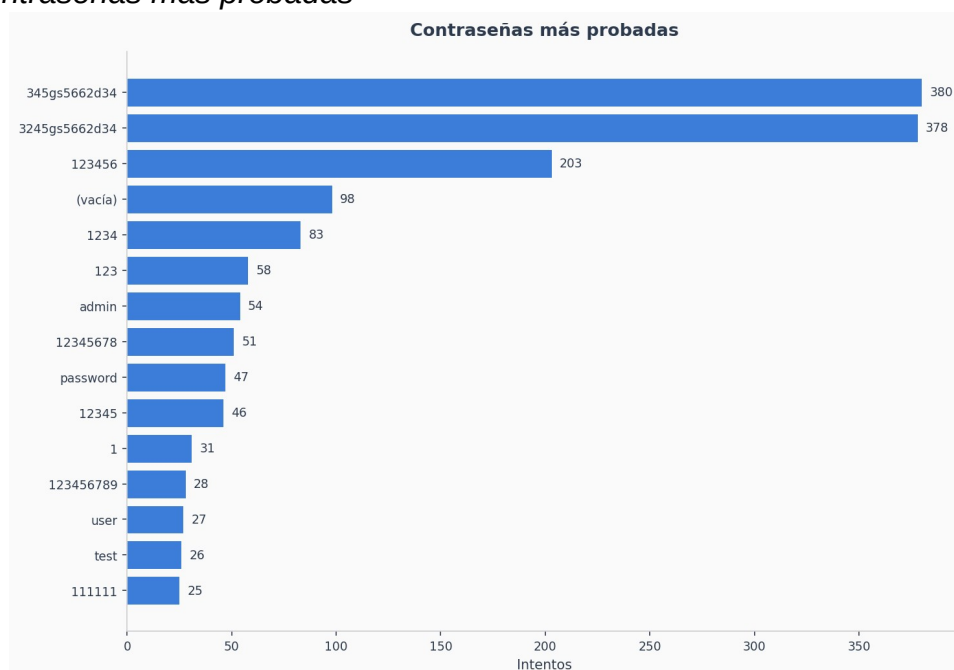
La geolocalización asigna a cada IP el país de su proveedor, que no tiene por qué coincidir con el del atacante: las IPs observadas pertenecen mayoritariamente a servidores alquilados y a equipos ya comprometidos que forman parte de botnets. Con esa salvedad, la Figura 6 muestra dos patrones opuestos. Países Bajos concentra el 72,7 % de las conexiones con solo 44 IPs, mientras que Estados Unidos (98 IPs) y China (74 IPs) son los países con más atacantes distintos, cada uno con poca actividad.

Figura 6*Conexiones por país del proveedor de la IP (diez primeros)*

El caso de Países Bajos se explica por una única campaña: 15 IPs consecutivas del rango 109.160.32.0/24, de un mismo proveedor de hosting, generan el 61,9 % de todas las conexiones del periodo (8 517). Probaron 8 491 combinaciones de usuario y contraseña sin repetir ninguna, repartiendo un diccionario entre las 15 direcciones para no superar umbrales de bloqueo por IP. Solo consiguieron entrar 9 veces y nunca ejecutaron comandos: su función es recolectar credenciales válidas.

Credenciales

El usuario más probado es root (1 860 intentos), seguido de admin, user, ubuntu y cuentas de servicios habituales (postgres, oracle, hadoop). Las contraseñas confirman el uso de diccionarios de credenciales triviales: 123456, 1234, admin, password y la contraseña vacía (98 intentos). Destaca la cadena 345gs5662d34 y su variante 3245gs5662d34, usadas 758 veces desde 93 IPs distintas: no es una contraseña que se intente adivinar, sino un valor fijo característico de una familia concreta de bots.

Figura 7**Contraseñas más probadas****Campañas Identificadas**

Los comandos y ficheros capturados permiten reconocer campañas documentadas públicamente. Los ficheros se identifican por su hash SHA-256, que es el indicador de compromiso que se compartiría con otros equipos de seguridad.

Campaña	Evidencia	Alcance	Objetivo
Botnet Outlaw (Dota)	Sustituye ~/.ssh por una clave propia con el comentario «mdrfckr» y la bloquea con chattr	396 sesiones, 97 IPs	Persistencia: acceso permanente aunque se cambie la contraseña
RedTail	Sube por SFTP setup.sh, clean.sh y el minero para cinco arquitecturas (x86_64, i686, ARMv7, ARMv8, RISC-V)	1 IP	Minería de criptomonedas con los recursos del servidor
Comprobación de honeypot	Script «filter» que analiza cómo responde la shell y recoge CPU, núcleos, GPU e historial de accesos	203 sesiones, 12 IPs	Decidir si la máquina es real y útil para minar
Toma de control	chpasswd sobre root; vaciado de /etc/hosts.deny y eliminación de procesos ajenos	15 y 14 sesiones	Expulsar al administrador y a otros intrusos
Uso como proxy	Peticiones de reenvío TCP, sobre todo hacia una IP en el puerto 2535	56 sesiones, 5 IPs	Ocultar el origen real del tráfico
Búsqueda de datos	Rutas de sesiones de Telegram Desktop y de	2 sesiones	Robo de cuentas o envío de SMS

Campaña	Evidencia	Alcance	Objetivo
	módems y pasarelas SMS		fraudulentos

Tabla 18. Campañas y comportamientos identificados.

La campaña de Outlaw es la más extendida: 97 IPs distintas repiten exactamente la misma secuencia de comandos, lo que indica una botnet ya consolidada que se propaga desde los equipos que infecta. Outlaw es una botnet de minería de Monero que entra por credenciales SSH débiles y cuyos scripts eliminan los mineros de la competencia (Souza et al., 2025). RedTail también es un minero, documentado por el SANS Internet Storm Center con exactamente el mismo conjunto de ficheros: un setup.sh que detecta la arquitectura de la víctima y ejecuta el binario correspondiente, además de instalar una clave SSH para garantizar la persistencia (Riley, 2024). En los datos del proyecto, el comando que acompaña a la subida incluye, efectivamente, la escritura de una clave en authorized_keys.

La comprobación de honeypot tiene una lectura directa para el proyecto: los atacantes también analizan el sistema al que acceden, y un honeypot de media interacción como Cowrie tiene límites para superar esas pruebas. Es una de las líneas de mejora recogidas en Trabajos Futuros.

Técnicas MITRE ATT&CK

El clasificador asignó la actividad a 21 técnicas de 9 tácticas. La Tabla 19 resume las más relevantes de cada táctica y la Figura 8 muestra todas ellas por número de sesiones.

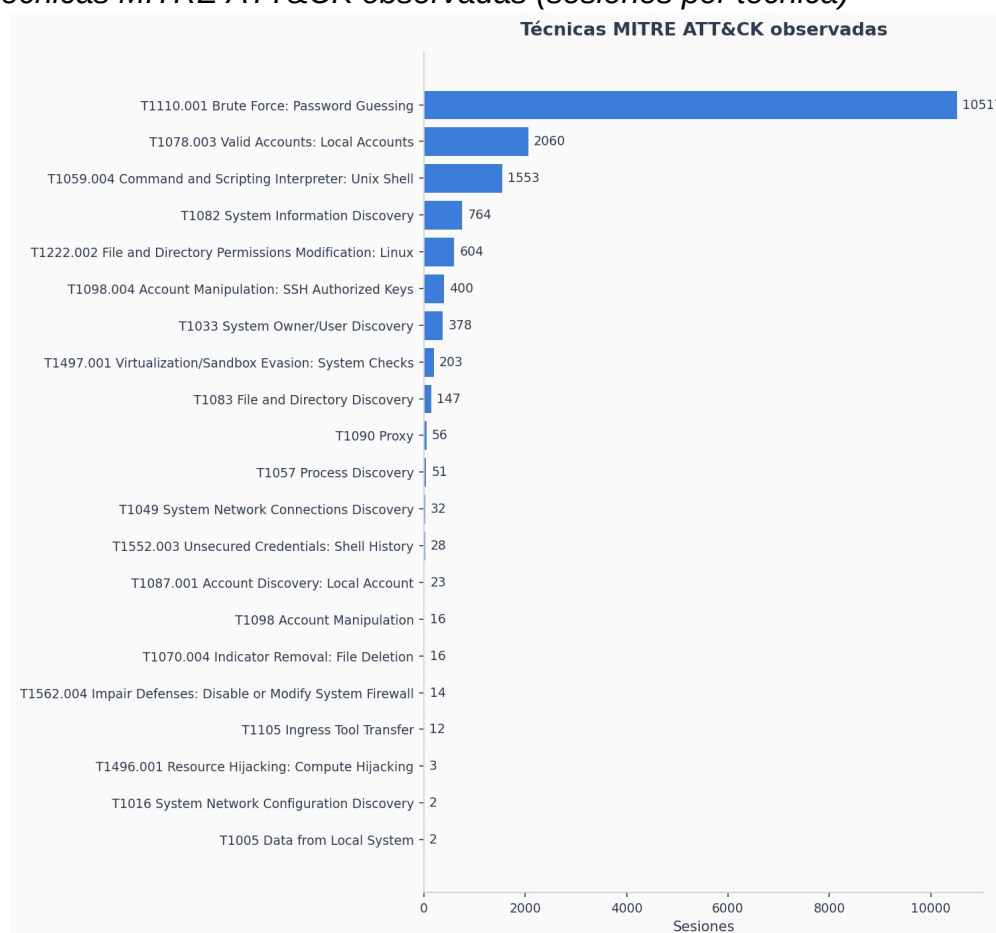
Táctica	Técnica	Sesiones
Credential Access	T1110.001 Brute Force: Password Guessing	10 517
Initial Access	T1078.003 Valid Accounts: Local Accounts	2 060
Execution	T1059.004 Command and Scripting Interpreter: Unix Shell	1 553
Discovery	T1082 System Information Discovery	764
Defense Evasion	T1222.002 File and Directory Permissions Modification	604
Persistence	T1098.004 Account Manipulation: SSH Authorized Keys	400
Defense Evasion	T1497.001 Virtualization/Sandbox Evasion: System Checks	203
Command and Control	T1090 Proxy	56
Command and Control	T1105 Ingress Tool Transfer	12

Táctica	Técnica	Sesiones
Impact	T1496.001 Resource Hijacking: Compute Hijacking	3
Collection	T1005 Data from Local System	2

Tabla 19. Técnicas MITRE ATT&CK más relevantes por táctica.

Figura 8

Técnicas MITRE ATT&CK observadas (sesiones por técnica)



Los datos reales obligaron a ampliar el clasificador. Siete técnicas no estaban previstas en las reglas iniciales (entre ellas T1497.001, T1090, T1005 y T1552.003), y se detectó un falso positivo: listar el directorio `/var/log` se clasificaba como borrado de registros (T1070.002). Tras los cambios, los comandos sin técnica asignada pasaron de 508 a 40 y el conjunto de pruebas automáticas creció de 33 a 50 casos, todos superados. También se corrigió el análisis de ficheros: Cowrie registra como descarga cualquier contenido escrito con una redirección (`echo ... > fichero`), de modo que las 623 «descargas» iniciales eran en su mayoría claves SSH y scripts escritos por los bots; ahora se agrupan por hash y se distingue el tipo de origen.

Primeras Conclusiones Defensivas

Casi toda la actividad observada se habría neutralizado con tres medidas que no requieren herramientas comerciales, y que coinciden con las mitigaciones de ATT&CK asociadas a cada técnica:

- **Autenticación SSH solo con clave y sin acceso como root (M1032, M1026):** anula la fuerza bruta, que supone el 76 % de las sesiones, y es la configuración aplicada al SSH real del proyecto en el puerto 2022.
- **Control de integridad sobre `authorized_keys` (M1022):** detectaría la persistencia de Outlaw, la campaña más extendida.
- **Filtrado del tráfico saliente y `AllowTcpForwarding` no (M1037, M1042):** impide la descarga de herramientas, la conexión del minero con su pool y el uso del servidor como proxy.

Estas conclusiones se basan en tres días de datos y se revisarán con el periodo completo de captura, ya que las campañas aparecen y desaparecen y algunas pueden no haberse manifestado todavía.

Trabajos Futuros

El proyecto sienta las bases para diversas líneas de ampliación y mejora que podrían desarrollarse como continuación de este trabajo, como proyecto de Máster o como implementación profesional en un entorno corporativo.

Evolución del Honeypot

- **Honeypot de alta interacción:** sustituir Cowrie por un sistema que proporcione un Linux real al atacante (mediante contenedores Docker o máquinas virtuales aisladas con snapshots automáticos), permitiendo capturar comportamientos más avanzados como escalada de privilegios, movimiento lateral, instalación de rootkits y comunicación con servidores de comando y control (C2). La configuración de Cowrie con backend proxy hacia una VM real ya lo soporta, aunque requiere un aislamiento riguroso.
- **Honeypot multi-protocolo:** ampliar la captura a otros servicios frecuentemente atacados: HTTP (ataques a paneles de administración, exploits de aplicaciones web), Telnet (IoT), FTP (credenciales y malware), SMB (ransomware, EternalBlue). Plataformas como T-Pot integran más de 20 honeypots diferentes en un único despliegue con Docker Compose, aunque requieren significativamente más recursos.
- **Red distribuida de honeypots:** desplegar instancias de Cowrie en múltiples regiones cloud (Europa, América, Asia) para comparar los patrones de ataque por zona geográfica y detectar campañas globales coordinadas. Los datos podrían centralizarse en un único dashboard para su correlación.

Mejora del Análisis

- **Análisis de malware:** integrar un sandbox como Cuckoo Sandbox o el servicio cloud de ANY.RUN para analizar automáticamente los binarios descargados por los atacantes, extrayendo indicadores de compromiso (IOC) adicionales: hashes, dominios contactados, claves de registro modificadas y comportamiento en red.
- **Machine learning para detección de patrones:** aplicar algoritmos de clustering (K-Means, DBSCAN) sobre las sesiones de ataque para identificar grupos de atacantes con comportamientos

similares, detectando campañas coordinadas o variantes de un mismo bot. Las características podrían incluir secuencias de comandos, intervalos temporales y credenciales probadas.

- **Visualización geográfica interactiva:** generar un mapa interactivo (Leaflet o Folium) con la distribución geográfica de los atacantes, actualizándose en tiempo real conforme se registran nuevos ataques. Incluir líneas de flujo que muestren las rutas de ataque y heatmaps de densidad por región.

Integración y Automatización

- **Automatización con Wazuh:** desarrollar reglas personalizadas y decodificadores en Wazuh que generen alertas en tiempo real cuando se detecten patrones de ataque específicos: más de 100 intentos de login desde la misma IP en una hora, descarga de binarios con hashes conocidos como malware, o ejecución de comandos asociados a técnicas MITRE ATT&CK concretas.
- **Pipeline SOAR:** construir un flujo de respuesta automatizada (Security Orchestration, Automation and Response) que, ante la detección de un ataque significativo, genere automáticamente el ticket correspondiente, notifique al equipo de seguridad y actualice las listas de bloqueo del firewall perimetral.
- **Alimentación de feeds de amenazas:** publicar las IPs atacantes, los hashes de malware y los IOC extraídos como feed de inteligencia en formato STIX/TAXII para su consumo por otros sistemas de seguridad (firewalls, IDS/IPS, SIEMs) y por la comunidad de ciberseguridad a través de plataformas como MISP (Malware Information Sharing Platform).

Conclusiones

Las conclusiones de este capítulo se basan en los resultados provisionales de los tres primeros días de captura y se revisarán con el periodo completo en la entrega final.

Conclusiones Técnicas

El despliegue de un honeypot SSH con Cowrie ha permitido verificar de primera mano la magnitud y la naturaleza de los ataques automatizados que recibe cualquier servidor expuesto a Internet. El primer ataque llegó 8 minutos después de exponer el puerto 22, y en los tres primeros días se registraron 13 761 conexiones desde 499 IPs. Este dato, aunque conocido en la literatura, adquiere una dimensión diferente cuando se observa en primera persona sobre infraestructura propia.

Los datos confirman que la actividad es casi exclusivamente automática: el primer comando llega en menos de un segundo tras la autenticación y el 85 % de los clientes son bibliotecas de programación. Se distinguen perfiles muy diferentes, desde la fuerza bruta masiva y distribuida, que recolecta credenciales sin llegar a usarlas, hasta botnets consolidadas como Outlaw, que buscan persistencia mediante claves SSH, y mineros como RedTail, preparados para cinco arquitecturas de procesador. También se observaron bots que comprueban si se encuentran en un honeypot, un recordatorio de que el defensor también es observado.

La clasificación MITRE ATT&CK identificó 21 técnicas en 9 tácticas, desde el acceso inicial con cuentas válidas (T1078.003) hasta el secuestro de recursos (T1496.001). La mayoría de las sesiones se concentran en las fases tempranas de la cadena de ataque, coherente con bots que buscan acceso rápido para minar criptomonedas o incorporar el servidor a una botnet. La conclusión defensiva más importante es que tres medidas básicas (SSH solo con clave y sin root, control de integridad de `authorized_keys` y filtrado del tráfico saliente) habrían neutralizado casi toda la actividad observada.

Conclusiones sobre la Infraestructura

Desde el punto de vista técnico, el proyecto demuestra que es posible montar una infraestructura de inteligencia de amenazas funcional con software exclusivamente libre y un coste de infraestructura muy

reducido. Un servidor pequeño (2 vCPU y 4 GB de RAM) es más que suficiente para ejecutar Cowrie, el sistema operativo y los servicios auxiliares simultáneamente. El cambio forzoso de Oracle Cloud a Hetzner Cloud demostró además el valor de automatizar el despliegue con scripts genéricos: cambiar de proveedor solo afectó a la creación de la máquina.

La combinación del firewall del proveedor (Cloud Firewall de Hetzner), el firewall del host (iptables) y la redirección NAT proporciona tres capas de protección para el SSH de administración real. El puerto 2022 está expuesto a Internet, pero al aceptar únicamente autenticación por clave y prohibir el acceso como root, los intentos de acceso no autorizados no tienen posibilidad de éxito. Esta arquitectura de defensa en profundidad es trasladable a cualquier entorno de producción.

El script Python desarrollado (`cowrie_analyzer.py`) automatiza el procesamiento de miles de eventos y genera artefactos visuales directamente utilizables en un informe de seguridad profesional. Su diseño modular permite ampliar el análisis con nuevas métricas sin modificar la estructura existente. El generador de datos de prueba (`generate_sample_logs.py`) ha permitido desarrollar y validar el script de análisis sin necesidad de esperar a la captura de datos reales, acelerando significativamente el desarrollo.

Conclusiones Formativas

Como reflexión profesional, el proyecto ha reforzado competencias transversales del ciclo de ASIR que son directamente aplicables en el mercado laboral:

- **Administración de sistemas Linux en entornos cloud:** despliegue de servidores, configuración de cortafuegos del proveedor, gestión de claves SSH y conexión remota en Hetzner Cloud, además del uso de la API de Oracle Cloud Infrastructure para automatizar el aprovisionamiento.
- **Seguridad de sistemas:** hardening de SSH, configuración de firewalls (iptables y Cloud Firewall de Hetzner), gestión de claves criptográficas y aplicación de buenas prácticas de seguridad recogidas en benchmarks CIS.
- **Scripting y automatización:** desarrollo de scripts Python para el procesamiento de datos JSON, generación de gráficas con matplotlib y consumo de APIs REST.

- **Análisis de logs de seguridad:** interpretación de eventos de seguridad, identificación de patrones de ataque y correlación de indicadores de compromiso.
- **Inteligencia de amenazas:** aplicación del marco MITRE ATT&CK para la clasificación de comportamientos adversarios y la formulación de recomendaciones defensivas.
- **Documentación técnica:** redacción de memoria de proyecto siguiendo estándares académicos, incluyendo diagramas, tablas y referencias bibliográficas.

Estas competencias son directamente aplicables en roles de administración de sistemas, analista SOC (Security Operations Center) y ciberseguridad en entornos empresariales. El proyecto demuestra capacidad para diseñar, desplegar, operar y documentar un sistema de seguridad completo de forma autónoma, desde la planificación hasta la entrega del informe final.

Valoración Personal

Esta valoración se escribe a mitad de proyecto, con el honeypot en marcha y los primeros datos analizados, y se completará en la entrega final.

La decisión que más me ha enseñado no fue técnica. Pasé más de un día esperando a que Oracle tuviera capacidad, con un script que lo intentaba cada minuto, y durante ese tiempo el proyecto estuvo bloqueado por una dependencia que no controlaba. Cuando por fin cambié de proveedor, el traslado solo afectó a la creación de la máquina, porque había escrito los scripts de despliegue sin atarlos a Oracle. De ahí saco dos aprendizajes: fijar un límite de tiempo a cualquier bloqueo externo antes de buscar alternativas, y diseñar desde el principio pensando en que el entorno puede cambiar. También rechacé la retención de 93 € que pedía Oracle para su plan de pago: preferí un proveedor con saldo prepago y sin tarjeta guardada, porque controlar el coste de un servidor expuesto a Internet también es una decisión de seguridad.

El segundo aprendizaje es desconfiar de lo que parece funcionar. Un carácter con tilde en el fichero de credenciales hacía que Cowrie rechazara todos los accesos sin mostrar ningún error, y el servicio de systemd fallaba por dos motivos encadenados que no aparecían en mis

pruebas locales. En ambos casos lo que me salvó fue haber obligado a los scripts a comprobar el resultado en lugar de dar los pasos por buenos. Lo mismo ocurrió con los datos: la primera gráfica de países atribuía el 90 % de los ataques a Países Bajos porque solo geolocalizaba las 20 IPs más activas. Corregirlo cambió la conclusión, y me recordó que un análisis puede ser técnicamente correcto y aun así engañoso.

Los resultados también me han dado una lección de humildad. Esperaba ver atacantes probando contraseñas y descubrí que una parte de ellos me estaba analizando a mí, comprobando si el servidor era un honeypot antes de decidir qué hacer. Observar en directo campañas documentadas como Outlaw o RedTail, en lugar de leer sobre ellas, da otra dimensión a lo estudiado en el ciclo.

Por último, el proyecto conecta con mi trayectoria anterior: tras muchos años como operador en una central receptora de alarmas, vigilar eventos, distinguir el ruido de lo relevante y actuar con un procedimiento claro me resulta familiar. Este trabajo me ha confirmado que quiero orientar mi carrera hacia la monitorización de seguridad y la respuesta a incidentes, y me ha dado un caso práctico real, documentado y público con el que demostrarlo.

Referencias

Según las normas APA 7.^a edición. Cada referencia se acompaña de un texto descriptivo indicando su uso en el proyecto.

Honeypot utilizado en el proyecto.

Cowrie Contributors. (s. f.). Cowrie SSH/Telnet Honeypot. GitHub.
<https://github.com/cowrie/cowrie>

Marco de clasificación de la actividad observada.

MITRE Corporation. (s. f.). MITRE ATT&CK. <https://attack.mitre.org/>

Plataforma cloud del honeypot: servidores y cortafuegos.

Hetzner Online GmbH. (s. f.). Hetzner Cloud.
<https://www.hetzner.com/cloud/>

Verificación de cuentas nuevas en la plataforma cloud.

Hetzner Online GmbH. (s. f.). Fraud prevention FAQ. Hetzner Docs.
<https://docs.hetzner.com/general/security-and-identify/fraud-prevention-faq/>

Plataforma cloud prevista en el anteproyecto.

Oracle Corporation. (s. f.). Oracle Cloud Free Tier.
<https://www.oracle.com/cloud/free/>

SDK utilizado por el script de aprovisionamiento oci_a1_retry.py.

Oracle. (s. f.-b). Oracle Cloud Infrastructure Python SDK.
<https://docs.oracle.com/en-us/iaas/tools/python/latest/>

Biblioteca SSH empleada en el ataque simulado de las pruebas.

Paramiko. (s. f.). Paramiko: A Python implementation of SSHv2.
<https://www.paramiko.org/>

Servicio de notificaciones push del script de aprovisionamiento.

ntfy. (s. f.). ntfy: Send push notifications to your phone or desktop via PUT/POST. <https://ntfy.sh/>

Límites del tier Always Free y política de recuperación de instancias ociosas.

Oracle. (s. f.-a). Always Free resources. Oracle Cloud Infrastructure Documentation.

https://docs.oracle.com/en-us/iaas/Content/FreeTier/freetier_topic-Always_Free_Resources.htm

SIEM para ingesta y visualización de logs.

Wazuh, Inc. (s. f.). Wazuh – Open Source Security Platform.

<https://wazuh.com/>

Biblioteca de generación de gráficas.

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95.

<https://doi.org/10.1109/MCSE.2007.55>

Geolocalización de IPs atacantes.

ip-api. (s. f.). IP Geolocation API. <https://ip-api.com/>

Referencia sobre ataques SSH y honeypots.

Provos, N. y Holz, T. (2007). *Virtual Honeypots: From Botnet Tracking to Intrusion Detection*. Addison-Wesley.

Informes de incidentes de seguridad en España.

Centro Criptológico Nacional. (2024). CCN-CERT IA – Informe Anual 2024. <https://www.ccn-cert.cni.es/>

Sistema operativo del servidor.

Canonical Ltd. (s. f.). Ubuntu Server 24.04 LTS. <https://ubuntu.com/server>

Distribución para pruebas de seguridad.

OffSec Services Limited. (s. f.). Kali Linux. <https://www.kali.org/>

Análisis de la botnet Outlaw, identificada en los resultados.

Souza, C., Muñoz, A. y Ovalle, E. (2025, 29 de abril). Outlaw cybergang attacking targets worldwide. Securelist (Kaspersky). <https://securelist.com/outlaw-botnet/116444/>

Análisis del minero RedTail, identificado en los resultados.

Riley, R. (2024, 22 de mayo). Analysis of "redtail" file uploads to ICS honeypot, a multi-architecture coin miner [Guest Diary]. SANS Internet Storm Center. <https://isc.sans.edu/diary/30950>

Anexos

Los anexos reproducen los ficheros de configuración y los scripts de despliegue del proyecto tal y como se encuentran en el directorio `config/` del repositorio (<https://github.com/diegofonterosa/PFC-Cowrie>).

Anexo A. Configuración de Cowrie (`cowrie.cfg`)

Fichero `/home/cowrie/honeypot/etc/cowrie.cfg`. Solo contiene las claves que difieren de los valores por defecto de Cowrie 3.1.0.

```
# =====
# cowrie.cfg – Configuración del honeypot (PFC-Cowrie)
# Proyecto Intermodular ASIR · Diego Pérez Fonterosa
#
# Cowrie 3.x carga primero sus valores por defecto (cowrie.cfg.dist)
# y después este fichero, así que aquí solo van las claves que cambian.
# Ruta en el servidor: /home/cowrie/honeypot/etc/cowrie.cfg
# =====

[honeypot]
# Nombre de host del sistema falso. El valor por defecto (svr04) es una
# firma conocida de Cowrie que algunos bots usan para detectar honeypots.
hostname = web-prod-02

# Un único fichero JSON en vez de uno por día: simplifica el análisis
# con cowrie_analyzer.py durante el periodo de captura.
logtype = plain

# Las descargas de los atacantes se guardan (para extraer sus hashes)
# con un límite de tamaño para no llenar el disco.
download_limit_size = 10485760

# Sesiones inactivas se cierran a los 3 minutos.
interactive_timeout = 180

[shell]
# Identidad del sistema falso, coherente con el filesystem que trae
# Cowrie (Debian 12). Se cambian los valores por defecto, también
# conocidos como firma, por otros plausibles de un Debian actualizado.
kernel_version = 6.1.0-31-amd64
kernel_build_string = #1 SMP PREEMPT_DYNAMIC Debian 6.1.128-1 (2025-02-07)
hardware_platform = x86_64
operating_system = GNU/Linux
# x86_64 es la arquitectura que más malware recibe: maximiza capturas.
arch = linux-x64-lsb

[ssh]
enabled = true
version = SSH-2.0-OpenSSH_9.2p1 Debian-2+deb12u5
# Cowrie escucha en 2222 (puerto sin privilegios). iptables redirige
# el tráfico del 22 público a este puerto.
listen_endpoints = tcp:2222:interface=0.0.0.0

[telnet]
```

```
# Fuera de alcance del proyecto.  
enabled = false  
  
[output_jsonlog]  
enabled = true  
logfile = ${honeypot:log_path}/cowrie.json  
epoch_timestamp = false
```

Anexo B. Credenciales del Honeypot (userdb.txt)

Fichero /home/cowrie/honeypot/etc/userdb.txt. Debe ser ASCII puro (véase el capítulo de Pruebas).

```
# =====
# userdb.txt - Credenciales aceptadas por el honeypot (PFC-Cowrie)
# Ruta en el servidor: /home/cowrie/honeypot/etc/userdb.txt
#
# IMPORTANTE: este fichero debe ser ASCII puro (sin tildes ni enies).
# Cowrie 3.x lo lee como ASCII; un solo caracter no ASCII hace que
# rechace TODOS los logins sin dar ningun aviso visible.
#
# Formato: usuario:x:password (gana la primera linea que coincide)
# '*' = cualquiera / '!' delante = denegar.
#
# Estrategia: se rechazan las passwords mas probadas (root, 123456,
# admin...) para que los bots sigan intentandolo y registren mas
# credenciales; cualquier otra password para root/admin/ubuntu entra,
# de modo que tambien se capturan los comandos tras el acceso.
# =====
root:x:!root
root:x:!123456
root:x:!password
root:x:*
admin:x:!admin
admin:x:*
ubuntu:x:!ubuntu
ubuntu:x:*
```

Anexo C. Configuración del SSH de Administración

Se instala como /etc/ssh/sshd_config.d/00-pfc-hardening.conf; el script sustituye __ADMIN_USER__ por el usuario administrador.

```
# =====
# /etc/ssh/sshd_config.d/00-pfc-hardening.conf
# SSH real de administracion (PFC-Cowrie)
#
# Se llama "00-" porque sshd usa el PRIMER valor que encuentra de cada
# opcion: asi este fichero gana a 50-cloud-init.conf, 60-cloudimg...
# =====

# Puerto no estandar: el 22 queda para el honeypot
Port 2022

# Solo claves publicas, nunca contraseñas
PubkeyAuthentication yes
PasswordAuthentication no
KbdInteractiveAuthentication no
AuthenticationMethods publickey

# Sin root y solo el usuario administrador (lo sustituye el script)
PermitRootLogin no
AllowUsers __ADMIN_USER__

# Limitar intentos y tiempo de login
MaxAuthTries 3
LoginGraceTime 30

# Funciones innecesarias desactivadas
X11Forwarding no
AllowAgentForwarding no
```

Anexo D. Servicio de systemd de Cowrie

Fichero /etc/systemd/system/cowrie.service.

```
# =====
# /etc/systemd/system/cowrie.service - Honeypot Cowrie (PFC-Cowrie)
# Arranca Cowrie en primer plano bajo systemd, como usuario sin
# privilegios y con el sistema de ficheros en solo lectura salvo var/.
# =====
[Unit]
Description=Cowrie SSH honeypot (PFC-Cowrie)
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=cowrie
Group=cowrie
WorkingDirectory=/home/cowrie/honeypot
# Se lanza twistd directamente y en primer plano (-n), con la salida al
# journal (journalctl -u cowrie). Es lo que hace "cowrie start" con
# COWRIE_STDOUT=yes, salvo dos detalles que lo rompan bajo systemd:
# - "cowrie start" busca twistd en el PATH y systemd no activa el venv
#   (FileNotFoundError): aqui va la ruta completa.
# - twistd escribe twistd.pid en el directorio de trabajo, que el
#   confinamiento deja en solo lectura: --pidfile= lo desactiva (systemd
#   ya sabe el PID del proceso).
ExecStart=/home/cowrie/honeypot/cowrie-env/bin/twistd --umask=0022 -n --
pidfile= --logger cowrie.python.logfile.stdoutLogger cowrie
Restart=always
RestartSec=10

# Confinamiento: aunque alguien escapase de Cowrie, no podria escribir
# fuera de var/ ni ganar privilegios.
NoNewPrivileges=yes
PrivateTmp=yes
PrivateDevices=yes
ProtectSystem=strict
ProtectHome=read-only
ReadWritePaths=/home/cowrie/honeypot/var
ProtectKernelTunables=yes
ProtectKernelModules=yes
ProtectControlGroups=yes
RestrictSUIDSGID=yes
CapabilityBoundingSet=
MemoryMax=512M

[Install]
WantedBy=multi-user.target
```

Anexo E. Reserva de Memoria (oci-keepalive)

Programa instalado en `/usr/local/lib/pfc-cowrie/` y su servicio en `/etc/systemd/system/`.

oci-keepalive.py

```
#!/usr/bin/env python3
"""
oci-keepalive.py - Evita que Oracle recupere la instancia por "ociosa"
Proyecto Intermodular ASIR - Diego Perez Fonterosa

Oracle puede recuperar una instancia Always Free si durante 7 dias tiene
a la vez CPU < 20 % (p95), red < 20 % y, en las A1, memoria < 20 %.
Un honeypot cumple las tres. Basta con romper una: este proceso reserva
un porcentaje fijo de la RAM y la mantiene ocupada, sin gastar CPU.

Porcentaje configurable con la variable KEEPALIVE_PERCENT (defecto 25).
"""
import os
import signal
import sys
import time

def meminfo(campo):
    with open("/proc/meminfo", encoding="ascii") as f:
        for linea in f:
            if linea.startswith(campo + ":"):
                return int(linea.split()[1]) * 1024 # kB -> bytes
        raise KeyError(campo)

def main():
    pct = float(os.environ.get("KEEPALIVE_PERCENT", "25"))
    if not 0 < pct <= 50:
        sys.exit("KEEPALIVE_PERCENT debe estar entre 0 y 50")

    total = meminfo("MemTotal")
    tam = int(total * pct / 100)

    # bytearray() pide paginas a cero que el kernel no asigna hasta que se
    # escriben: hay que tocar cada pagina para que cuenten como RAM usada.
    buf = bytearray(tam)
    pagina = os.sysconf("SC_PAGE_SIZE")
    for i in range(0, tam, pagina):
        buf[i] = 1

    print(f"Reservados {tam / 2**20:.0f} MiB ({pct:g} % de {total / 2**30:.1f} GiB)", flush=True)

    signal.signal(signal.SIGTERM, lambda _: sys.exit(0))
    while True:
```

```

        time.sleep(3600)

if __name__ == "__main__":
    main()

```

oci-keepalive.service

```

# =====
# /etc/systemd/system/oci-keepalive.service (PFC-Cowrie)
# Reserva el 25 % de la RAM para que Oracle no considere ociosa la
# instancia A1 (politica de recuperacion de instancias Always Free).
# =====
[Unit]
Description=Reserva de memoria anti-recuperacion de instancias ociosas OCI
(PFC-Cowrie)
After=multi-user.target

[Service]
Type=simple
Environment=KEEPALIVE_PERCENT=25
ExecStart=/usr/bin/python3 /usr/local/lib/pfc-cowrie/oci-keepalive.py
Restart=always
RestartSec=30
Nice=19

# La reserva tiene que estar en RAM: si se fuese a swap no contaria.
MemorySwapMax=0

# Confinamiento: usuario efimero sin privilegios ni acceso a ficheros
DynamicUser=yes
NoNewPrivileges=yes
ProtectSystem=strict
ProtectHome=yes
PrivateTmp=yes
PrivateDevices=yes
PrivateNetwork=yes
CapabilityBoundingSet=
RestrictAddressFamilies=AF_UNIX

[Install]
WantedBy=multi-user.target

```

Anexo F. Script de Bastionado (01_hardening.sh)

```
#!/usr/bin/env bash
# =====
# 01_hardening.sh - Bastionado del servidor (PFC-Cowrie)
# Proyecto Intermodular ASIR - Diego Perez Fonterosa
#
# - Actualiza el sistema y activa las actualizaciones automaticas
# - Mueve el SSH real al puerto 2022, solo con clave y sin root
# - Firewall con iptables (persistente): denegar todo salvo lo necesario
# - Crea swap si la maquina tiene menos de 2 GB de RAM
#
# Uso: sudo bash 01_hardening.sh (o como root: bash 01_hardening.sh)
# Probado para Ubuntu Server 24.04 LTS. Valido para proveedores que
# entregan un usuario "ubuntu" (Oracle, AWS) o solo root (Hetzner).
# =====
set -euo pipefail

SSH_PORT=2022
SCRIPT_DIR="$(cd "$(dirname "$0")" && pwd)"
DROPIN=/etc/ssh/sshd_config.d/00-pfc-hardening.conf

info() { echo -e "\n\033[1;34m[+] \033[0m $*"; }
warn() { echo -e "\033[1;33m[!] \033[0m $*"; }
die() { echo -e "\033[1;31m[x] \033[0m $*" >&2; exit 1; }

# --- Comprobaciones previas -----
[[ $EUID -eq 0 ]] || die "Ejecuta el script con sudo (o como root)."
[[ -f "$SCRIPT_DIR/sshd_hardening.conf" ]] || die "Falta sshd_hardening.conf
junto al script."

# --- Usuario administrador -----
# Este script prohíbe el acceso SSH como root, así que antes debe existir
# un usuario administrador con tu clave. Si la maquina solo trae root
# (Hetzner), se crea uno copiando la clave autorizada de root.
if [[ -n "${SUDO_USER:-}" && "${SUDO_USER}" != "root" ]]; then
    ADMIN_USER="$SUDO_USER"
elif id ubuntu &>/dev/null && [[ -s /home/ubuntu/.ssh/authorized_keys ]]; then
    ADMIN_USER=ubuntu
else
    [[ -s /root/.ssh/authorized_keys ]] || die "root no tiene claves en
/root/.ssh/authorized_keys."
    read -rp " Nombre del usuario administrador a crear [diego]: " ADMIN_USER
    ADMIN_USER="${ADMIN_USER:-diego}"
    [[ "$ADMIN_USER" =~ ^[a-z_][a-z0-9_-]*$ ]] || die "Nombre de usuario no
valido."
    if ! id "$ADMIN_USER" &>/dev/null; then
        info "Creando el usuario administrador $ADMIN_USER..."
        adduser --disabled-password --gecos "" "$ADMIN_USER"
    fi
    usermod -aG sudo "$ADMIN_USER"
    install -d -m 700 -o "$ADMIN_USER" -g "$ADMIN_USER" "/home/$ADMIN_USER/.ssh"
    install -m 600 -o "$ADMIN_USER" -g "$ADMIN_USER"
/root/.ssh/authorized_keys \
    "/home/$ADMIN_USER/.ssh/authorized_keys"
```

```

# Solo se pide contrasena si el usuario aun no tiene una (estado "P"),
# para poder volver a ejecutar el script sin repetir este paso.
if [[ "${passwd -S "$ADMIN_USER" | awk '{print $2}'}" != "P" ]]; then
    echo " Elige la contrasena de $ADMIN_USER. Solo la pedira sudo: el SSH
seguira siendo solo con clave."
    passwd "$ADMIN_USER"
fi
fi
[[ -s "/home/$ADMIN_USER/.ssh/authorized_keys" ]] \
|| die "El usuario $ADMIN_USER no tiene claves en ~/.ssh/authorized_keys: te
quedarias fuera."
info "Usuario administrador: $ADMIN_USER"

cat <<EOF

ANTES DE CONTINUAR, en el firewall de tu proveedor cloud abre la
entrada TCP $SSH_PORT desde cualquier origen (0.0.0.0/0):
- Hetzner: Firewalls > (tu firewall) > Reglas de entrada
- Oracle: Networking > VCN > Subnet > Security List > Ingress Rule
- AWS:     EC2 > Security Groups > Inbound rules

Si el puerto $SSH_PORT no esta abierto ahi, perderas el acceso.
EOF
read -rp " El puerto $SSH_PORT ya esta abierto en el firewall del proveedor?
(si/no): " ok
[[ "$ok" == "si" ]] || die "Abrelo primero y vuelve a ejecutar el script."

# --- 1. Sistema -----
info "Actualizando el sistema..."
export DEBIAN_FRONTEND=noninteractive
apt-get update -q
apt-get upgrade -yq
apt-get install -yq iptables-persistent unattended-upgrades
dpkg-reconfigure -f noninteractive unattended-upgrades

# --- 2. Swap (instancias pequenas, p. ej. E2.1.Micro con 1 GB) -----
mem_mb=$(awk '/MemTotal/ {print int($2/1024)}' /proc/meminfo)
if (( mem_mb < 2048 )) && ! swapon --show | grep -q .; then
    info "RAM de ${mem_mb} MB: creando 2 GB de swap..."
    fallocate -l 2G /swapfile && chmod 600 /swapfile
    mkswap /swapfile && swapon /swapfile
    grep -q '^/swapfile' /etc/fstab || echo '/swapfile none swap sw 0 0' >>
/etc/fstab
fi

# --- 3. Firewall (iptables) -----
# Se usa iptables y no UFW: las imagenes Ubuntu de Oracle ya traen reglas
# en /etc/iptables/rules.v4 (incluidas las de los servicios internos de
# OCI en 169.254.0.0/16) y UFW gestionaria las mismas tablas en paralelo.
info "Configurando el firewall..."
if ufw status 2>/dev/null | grep -q "Status: active"; then
    warn "UFW estaba activo: se desactiva para no duplicar reglas."
    ufw disable
fi

ensure() { # inserta la regla al principio solo si no existe ya
    local chain=$1; shift
    iptables -C "$chain" "$@" 2>/dev/null || iptables -I "$chain" 1 "$@"

```

```

}

# Si la cadena INPUT no tiene ya un "denegar todo" (p. ej. Hetzner o AWS),
# se crea la base: trafico establecido, loopback, ICMP, 22 y rechazo final.
if ! iptables -S INPUT | grep -qE -- '-j (REJECT|DROP)$|-j REJECT --reject-
with'; then
    iptables -A INPUT -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
    iptables -A INPUT -i lo -j ACCEPT
    iptables -A INPUT -p icmp -j ACCEPT
    iptables -A INPUT -p tcp --dport 22 -m conntrack --ctstate NEW -j ACCEPT
    iptables -A INPUT -j REJECT --reject-with icmp-host-prohibited
fi
ensure INPUT -p tcp --dport "$SSH_PORT" -m conntrack --ctstate NEW -j ACCEPT
netfilter-persistent save

# --- 4. SSH -----
info "Configurando SSH en el puerto $SSH_PORT..."
# "Port" se acumula entre ficheros: si sshd_config fija el 22, se comenta.
sed -i -E 's/^\s*Port\s+/#&/' /etc/ssh/sshd_config
sed "s/__$ADMIN_USER__$ADMIN_USER/" "$SCRIPT_DIR/sshd_hardening.conf" >
"$DROPIN"
chmod 644 "$DROPIN"

# Ubuntu 24.04 arranca sshd por socket y /run/sshd solo existe mientras el
# servicio corre; "sshd -t" falla sin ella aunque la configuracion este bien.
install -d -m 0755 /run/sshd
if ! sshd -t; then
    rm -f "$DROPIN"
    die "La configuracion de sshd no es valida; se deshace el cambio."
fi

# Ubuntu 24.04 arranca SSH por socket: el puerto lo fija ssh.socket.
systemctl daemon-reload
if systemctl is-enabled ssh.socket &>/dev/null; then
    systemctl restart ssh.socket
fi
systemctl restart ssh.service

sleep 2
ss -ltn "sport = :$SSH_PORT" | grep -q LISTEN \
|| die "sshd no escucha en $SSH_PORT. NO cierres esta sesion y revisa:
journalctl -u ssh"

info "Hecho. SSH escuchando en el puerto $SSH_PORT."
cat <<EOF

SIGUIENTE PASO (sin cerrar esta sesion):
    abre OTRA terminal y conecta con
        ssh -p $SSH_PORT -i <tu_clave> $ADMIN_USER@<IP_PUBLICA>
    Si entra, continua con: sudo bash 02_instalar_cowrie.sh

EOF

```

Anexo G. Script de Instalación de Cowrie (02_instalar_cowrie.sh)

```
#!/usr/bin/env bash
# =====
# 02_instalar_cowrie.sh - Instalacion del honeypot (PFC-Cowrie)
# Proyecto Intermodular ASIR - Diego Perez Fonterosa
#
# - Crea el usuario sin privilegios "cowrie"
# - Instala Cowrie 3.1.0 en un entorno virtual de Python
# - Aplica cowrie.cfg y userdb.txt del proyecto
# - Lo deja como servicio de systemd escuchando en el puerto 2222
# - En instancias A1, activa oci-keepalive (reserva de memoria para que
#   Oracle no recupere la instancia por considerarla ociosa)
#
# Uso: sudo bash 02_instalar_cowrie.sh (despues de 01_hardening.sh)
# =====
set -euo pipefail

COWRIE_VERSION=3.1.0
HP_DIR=/home/cowrie/honeypot
SCRIPT_DIR="$(cd "$(dirname "$0")" && pwd)"

info() { echo -e "\n\033[1;34m[+] \033[0m $*"; }
die() { echo -e "\033[1;31m[x] \033[0m $*" >&2; exit 1; }

[[ $EUID -eq 0 ]] || die "Ejecuta el script con sudo."
for f in cowrie.cfg userdb.txt cowrie.service; do
    [[ -f "$SCRIPT_DIR/$f" ]] || die "Falta $f junto al script."
done

# userdb.txt con un solo caracter no ASCII hace que Cowrie rechace todos
# los logins sin avisar (comprobado en pruebas): se valida antes.
if LC_ALL=C grep -qP '^[^x00-^x7F]' "$SCRIPT_DIR/userdb.txt"; then
    die "userdb.txt contiene caracteres no ASCII (tildes, enies...). Quitalos."
fi

info "Instalando dependencias..."
export DEBIAN_FRONTEND=noninteractive
apt-get install -y python3-venv python3-pip python3-dev libssl-dev libffi-dev
build-essential

info "Creando el usuario cowrie..."
id cowrie &>/dev/null || adduser --disabled-password --gecos "" cowrie

info "Instalando Cowrie $COWRIE_VERSION..."
sudo -u cowrie -H bash -s <<EOF
set -euo pipefail
mkdir -p "$HP_DIR" && cd "$HP_DIR"
[[ -d cowrie-env ]] || python3 -m venv cowrie-env
source cowrie-env/bin/activate
python -m pip install -q --upgrade pip
python -m pip install -q "cowrie==$COWRIE_VERSION"
[[ -f etc/cowrie.cfg ]] || cowrie init
# Cache de plugins de Twisted: bajo systemd el venv es de solo lectura y,
```

```

# sin generarla aqui, twistd avisaria en cada arranque de que no puede
escribirla.
python -c "from twisted.plugin import IPlugin, getPlugins;
list(getPlugins(IPlugin))"
EOF

info "Aplicando la configuracion del proyecto..."
install -o cowrie -g cowrie -m 640 "$SCRIPT_DIR/cowrie.cfg"
"$HP_DIR/etc/cowrie.cfg"
install -o cowrie -g cowrie -m 640 "$SCRIPT_DIR/userdb.txt"
"$HP_DIR/etc/userdb.txt"

info "Creando el servicio de systemd..."
install -m 644 "$SCRIPT_DIR/cowrie.service" /etc/systemd/system/cowrie.service
systemctl daemon-reload
systemctl enable --now cowrie.service

sleep 8
systemctl is-active --quiet cowrie || die "Cowrie no arranca. Revisa:
journalctl -u cowrie -n 50"
ss -ltn "sport = :2222" | grep -q LISTEN || die "Cowrie no escucha en 2222.
Revisa: journalctl -u cowrie"

info "Cowrie instalado y escuchando en el puerto 2222."

# --- Reserva de memoria (solo instancias A1 de Oracle) -----
# Oracle recupera instancias Always Free con CPU, red y memoria (en A1)
# por debajo del 20 % durante 7 dias. Un honeypot cumple las tres; con
# mantener la memoria por encima del 20 % deja de considerarse ociosa.
mem_mb=$(awk '/MemTotal/ {print int($2/1024)}' /proc/meminfo)
es_oci=$(cat /sys/class/dmi/id/chassis_asset_tag 2>/dev/null || true)
if [[ "$es_oci" == "OracleCloud.com" && "$(uname -m)" == "aarch64" ]] &&
(( mem_mb >= 2048 )); then
    [[ -f "$SCRIPT_DIR/oci-keepalive.py" && -f "$SCRIPT_DIR/oci-
keepalive.service" ]] \
        || die "Faltan oci-keepalive.py u oci-keepalive.service junto al script."
    info "Instancia A1 con ${mem_mb} MB: activando la reserva de memoria (25
%)..."
    install -D -m 644 "$SCRIPT_DIR/oci-keepalive.py"
/usr/local/lib/pfc-cowrie/oci-keepalive.py
    install -m 644 "$SCRIPT_DIR/oci-keepalive.service" /etc/systemd/system/oci-
keepalive.service
    systemctl daemon-reload
    systemctl enable --now oci-keepalive.service
    sleep 5
    systemctl is-active --quiet oci-keepalive || die "oci-keepalive no arranca:
journalctl -u oci-keepalive"
    free -h | awk 'NR<=2'
else
    info "No es una instancia A1 de Oracle: la reserva de memoria no hace
falta."
fi
cat <<EOF

PRUEBA (desde tu equipo o Kali, todavia por el puerto 2222):
1. En el firewall del proveedor abre TEMPORALMENTE el 2222/TCP
2. ssh -p 2222 root@<IP_PUBLICA>      (password "root" -> rechazada)
   ssh -p 2222 root@<IP_PUBLICA>      (password "prueba" -> entra)

```

3. Ejecuta "uname -a" y sal. En el servidor:
 sudo tail -n 5 \$HP_DIR/var/log/cowrie/cowrie.json
4. Cierra el 2222 en el firewall del proveedor y continua con:
 sudo bash 03_redireccion_22.sh

EOF

Anexo H. Script de Redirección (03_redireccion_22.sh)

```
#!/usr/bin/env bash
# =====
# 03_redireccion_22.sh - Exponer el honeypot en el puerto 22
# Proyecto Intermodular ASIR - Diego Perez Fonterosa
#
# Redirige (NAT) las conexiones entrantes al 22 hacia el 2222 de
# Cowrie. Es el ultimo paso: a partir de aqui el honeypot es publico.
#
# Uso: sudo bash 03_redireccion_22.sh
# =====
set -euo pipefail

info() { echo -e "\n\033[1;34m[+] \033[0m $*"; }
die() { echo -e "\033[1;31m[x] \033[0m $*" >&2; exit 1; }

[[ $EUID -eq 0 ]] || die "Ejecuta el script con sudo."

# Seguridad: no redirigir el 22 si el SSH real no esta ya en el 2022
# o si Cowrie no esta escuchando.
ss -ltn "sport = :2022" | grep -q LISTEN || die "El SSH real no escucha en
2022. Ejecuta antes 01_hardening.sh."
systemctl is-active --quiet cowrie || die "Cowrie no esta activo.
Ejecuta antes 02_instalar_cowrie.sh."

ensure() { # inserta la regla solo si no existe ya
    local table=$1 chain=$2; shift 2
    iptables -t "$table" -C "$chain" "$@" 2>/dev/null || iptables -t "$table" -I
"$chain" 1 "$@"
}

info "Aplicando la redireccion 22 -> 2222..."
# Tras PREROUTING el paquete llega a INPUT con destino 2222, por eso el
# firewall del host debe aceptar el 2222 aunque el proveedor solo exponga el
22.
ensure filter INPUT -p tcp --dport 2222 -m conntrack --ctstate NEW -j ACCEPT
ensure nat PREROUTING -p tcp --dport 22 -j REDIRECT --to-port 2222
netfilter-persistent save

info "Reglas activas:"
iptables -t nat -S PREROUTING | grep 2222
iptables -S INPUT | grep -E -- '--dport (2022|2222)'

cat <<EOF

    El honeypot ya es publico en el puerto 22.
    Comprobacion desde fuera: ssh root@<IP_PUBLICA> -> debe verse web-prod-02
    Administracion: ssh -p 2022 ${SUDO_USER:-
<tu_usuario>}@<IP_PUBLICA>
    Ver ataques en directo: sudo tail -f
/home/cowrie/honeypot/var/log/cowrie/cowrie.json

EOF
```